Channel Coding with Racket

- About
- Contact
- Miscellaneous test code
- source: Binary BCH, Reed-Solomon, Golay, Product codes
- Source: Hamming, Reed-Muller, Cyclic, and general linear block code properties
- Things to do
- Update
- Usage reference

Channel coding suite

Tumblr ↗️

- This time, ODT file with no headers (sorry about the goof-up last time)

I just copy-pasted this ODT onto a fresh .rkt file in DrRacket and it compiles.

A far more usable version of the Racket code in this ODT should appear at my recently created GitHub account pretty soon. Here is my GitHub account: https://github.com/SathyaBhaskaran

Attached file

AllRacketCodeDocNoHeadersDownload

all_racket_code_18_DEC_2018Download

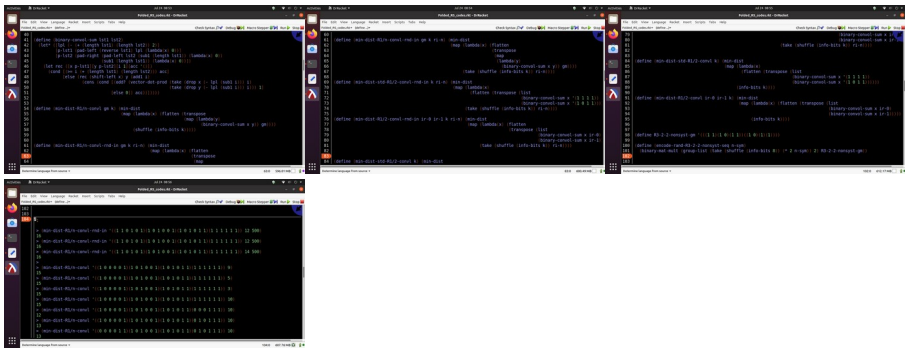
- How to write 10 convolutional encoders in one sitting (or forty)

I'm somewhat thrilled to announce that we now have a possible candidate for the first fascicle. It's going to be cyclic convolutional codes. You need to surmise the rest. You get 3 surmises. Due credit to an old Dennis the Menace in the local newspaper years ago for that surmising game gem.
On to the screenshots. Not yet. They're at the end of the post.

Also, I found some excellent papers on cyclic convolutional codes. The best of them was a two-page German-Dutch collab between TUM-Groningen-Oldenburg (Ossietzky).

Also also, in recent months I got sidetracked by an obsessive need to revisit my electronics fundamentals, in part thanks to the India Semiconductor Mission. In particular, this 1600-odd page free optoelectronics handbook has probably the best introduction to electromagnetics I've read. And I've read David Cheng.

To prove that I'm the same guy, I'm posting this (post) in the channelcodingwithrkt site.



- Sponsor a project to document this work and make it more usable

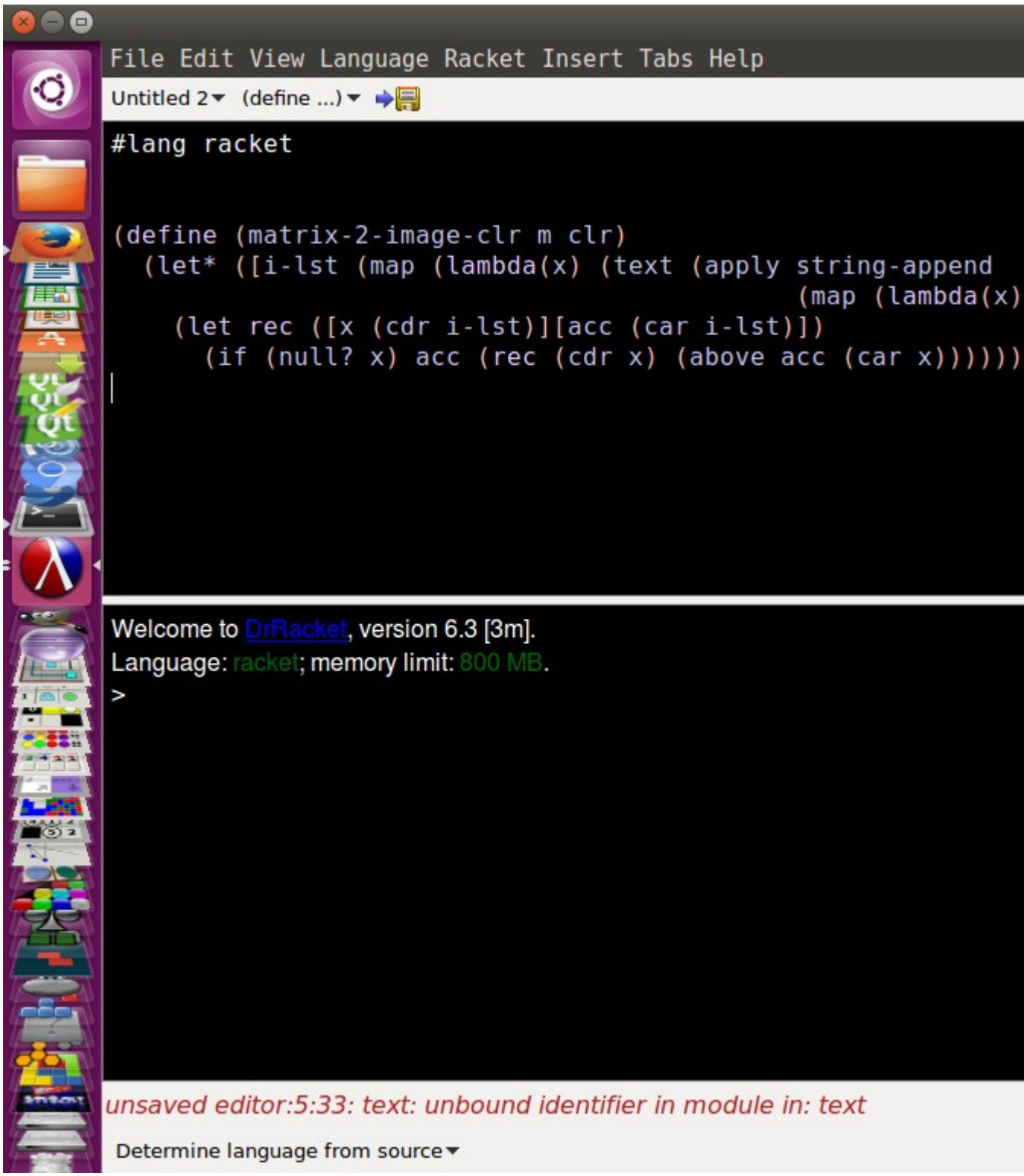
1. Till now the most used functions on this suite tend to be the relatively better documented ones, but there are hundreds of other functions that will be of interest to researchers. Most of these undocumented functions are incomplete. Many future features like squaring construction depend on their completion and can easily spawn several PhD's over a multi-year timeline.
2. The project can be a quasi-academic one because it's best to involve at least one of the Racketeers (assuming they'll be interested in such a project). This is way outside their programming language research area but this suite has the potential to knock out a certain spectacularly expensive commercial suite. As such the potential is so good, matching the aforesaid commercial suite feature for feature would be aiming low.

3. There's little point continuing to build convolutional codes alone. I'm exhausted from 7 years of working alone and would certainly jump at any opportunity for collaboration.

Okay, three points are good enough.

- Building on previous post's code

Use 2htdp/universe function run-movie with varying delay values. Blue line could be 0, red can be 1.
For encoder input use the following matrix->image function:



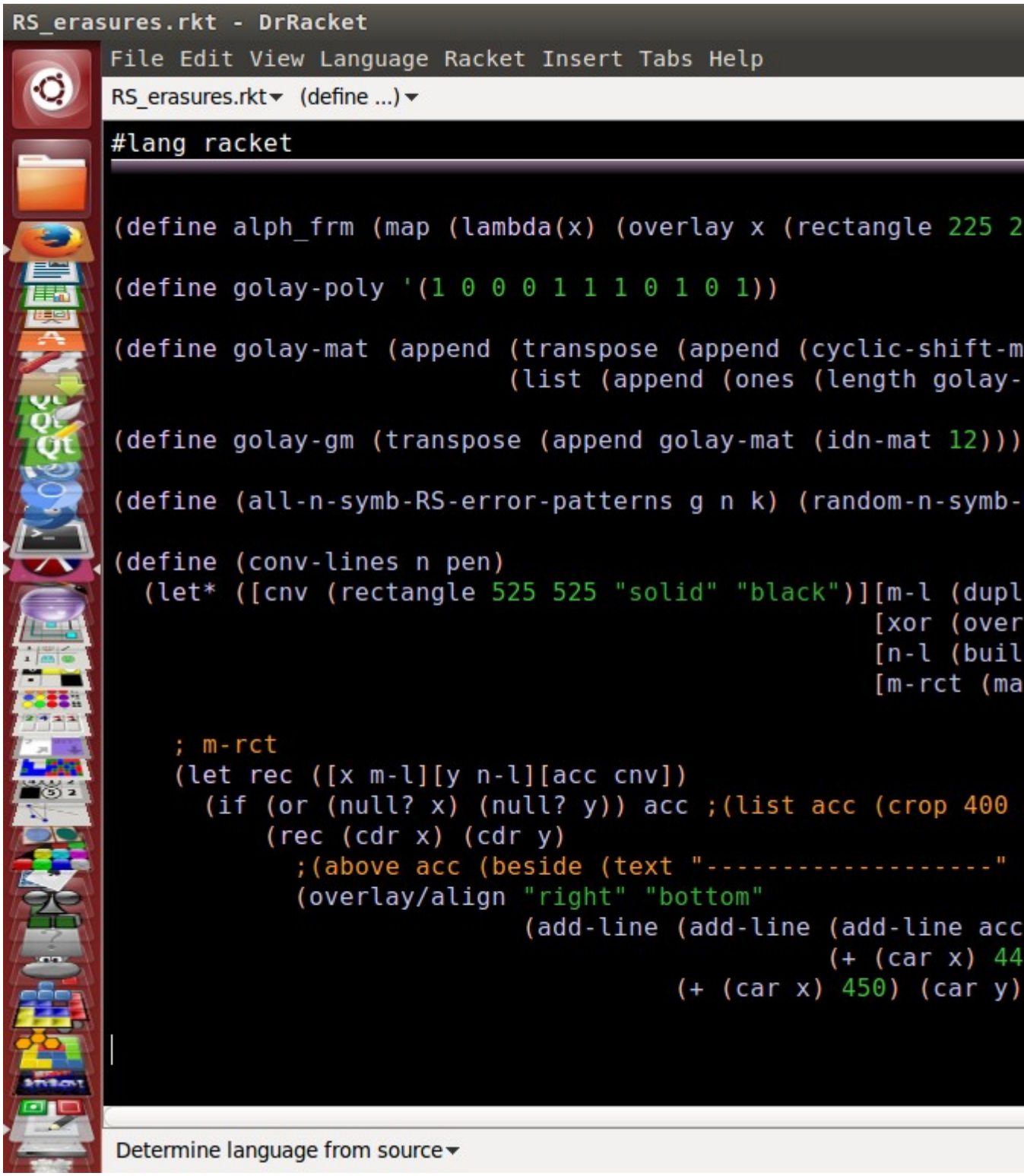
Here’s an array of non-binary samples you can feed to the run-movie function in 2htdp/universe



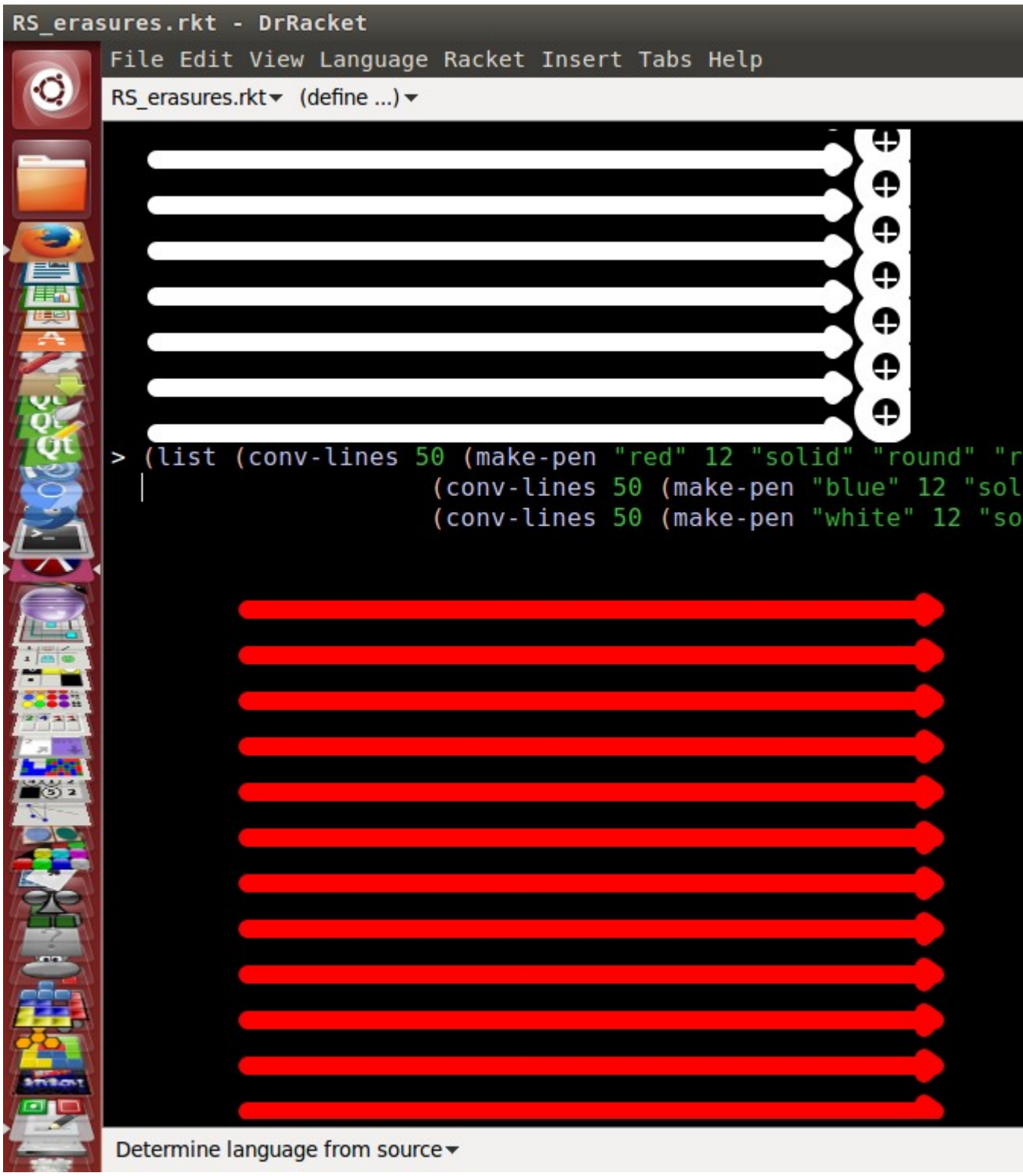
A more sophisticated apparoach is draw a rectangular grid of connectors. Code up only the building blocks, then rotate them and make a canvas etc. I'll explain this in more detail nexr.

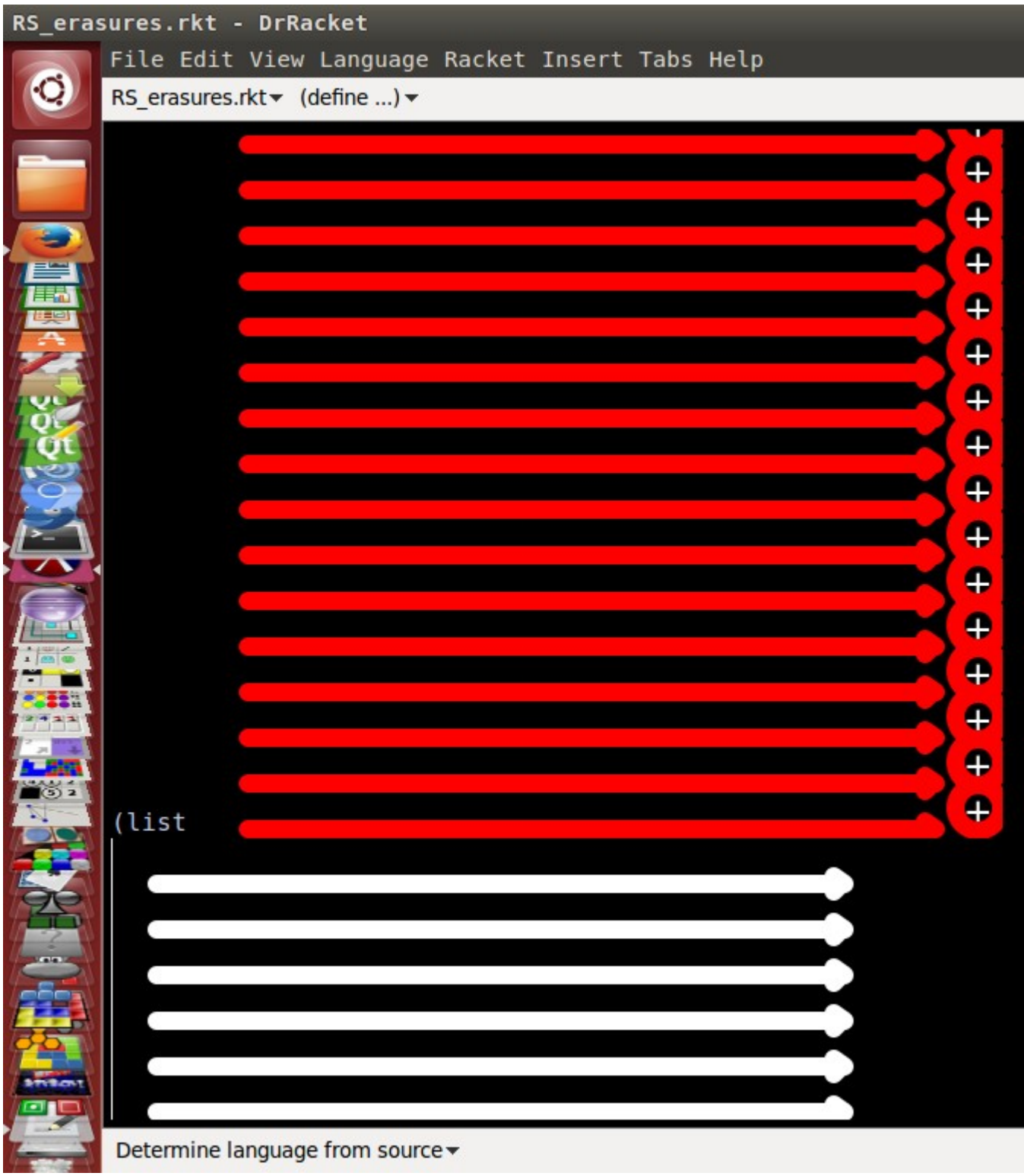
- Extremely boring progress with animating convolutional encoder

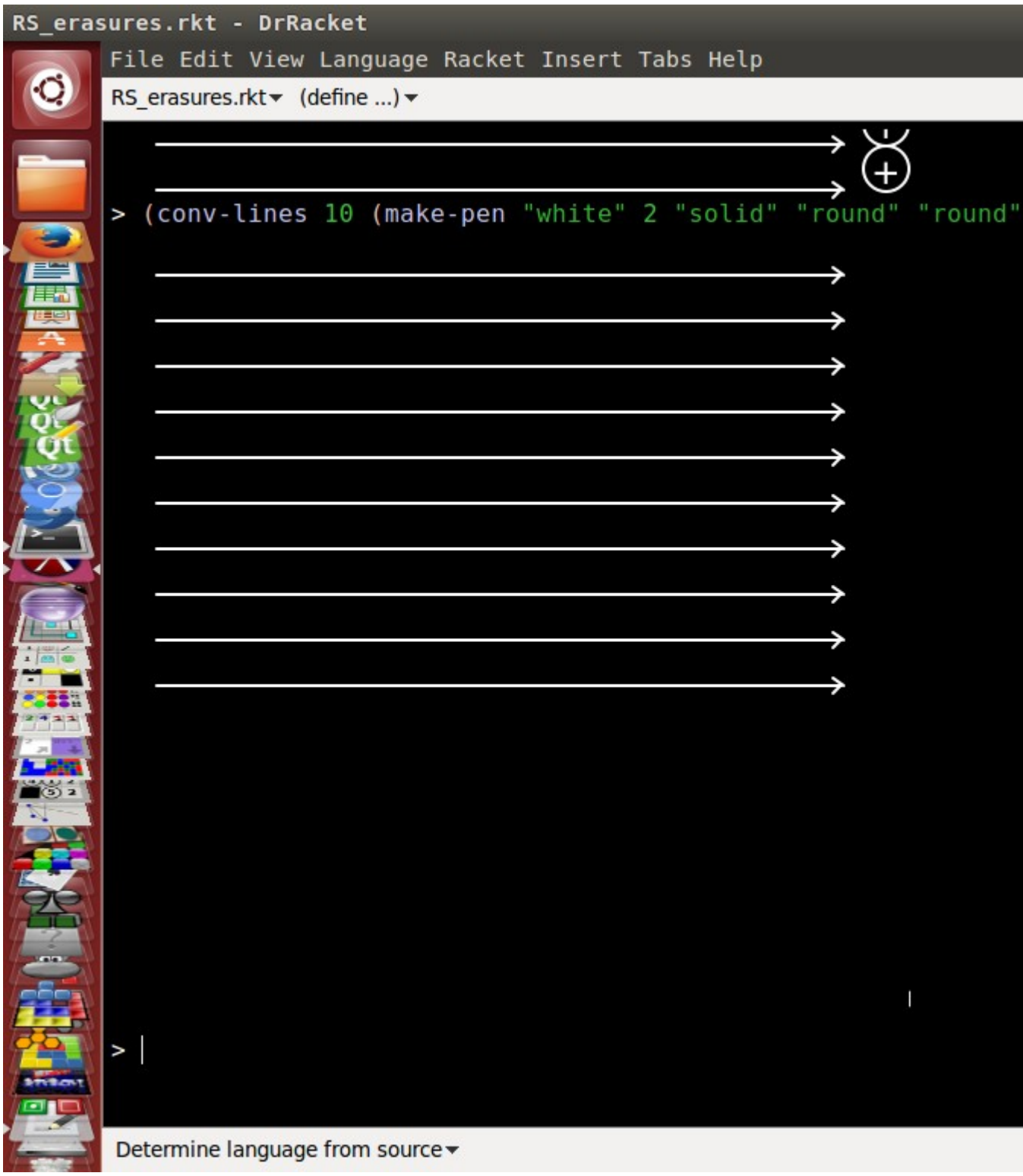
Here's some code you can run straight away:

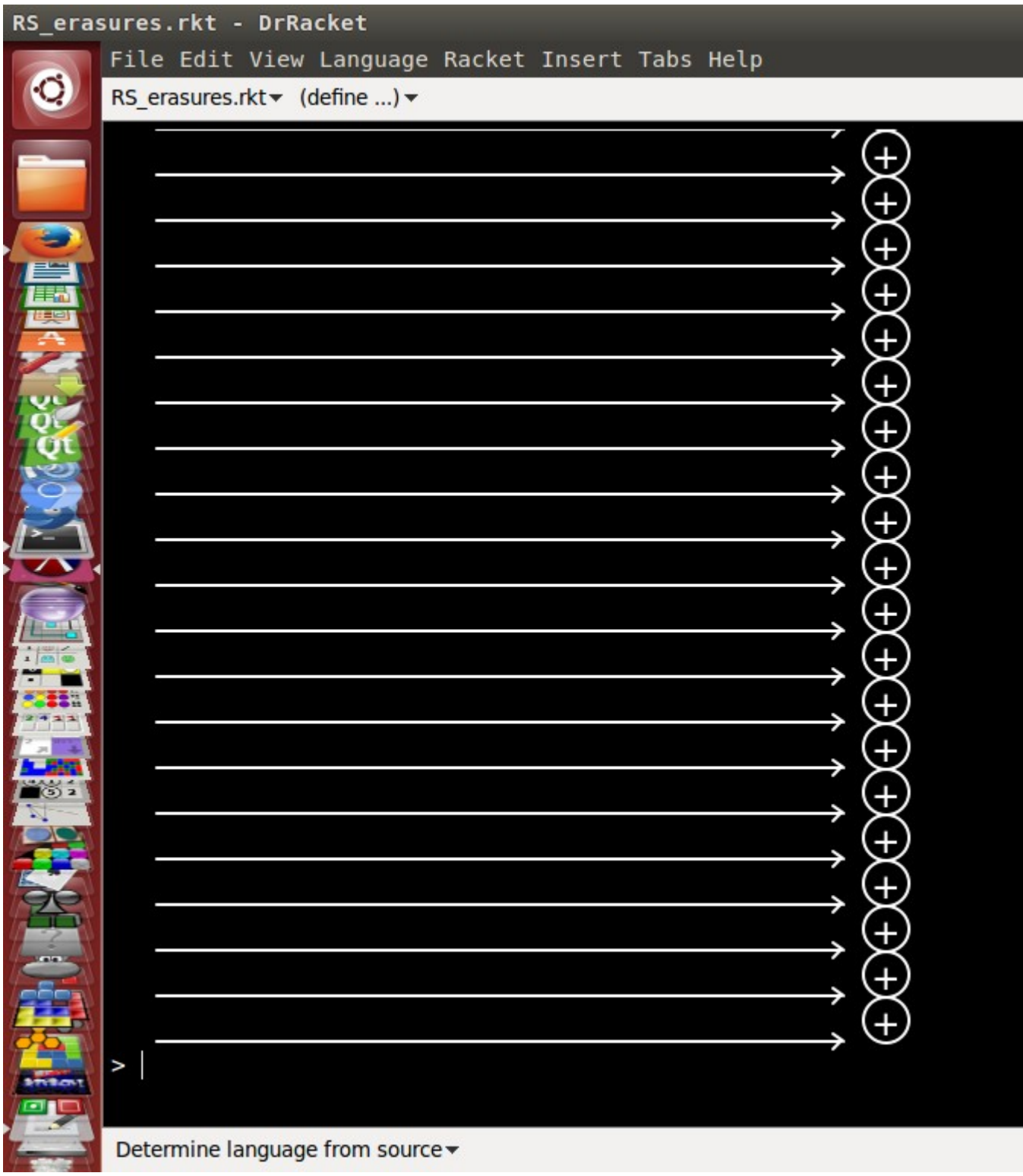


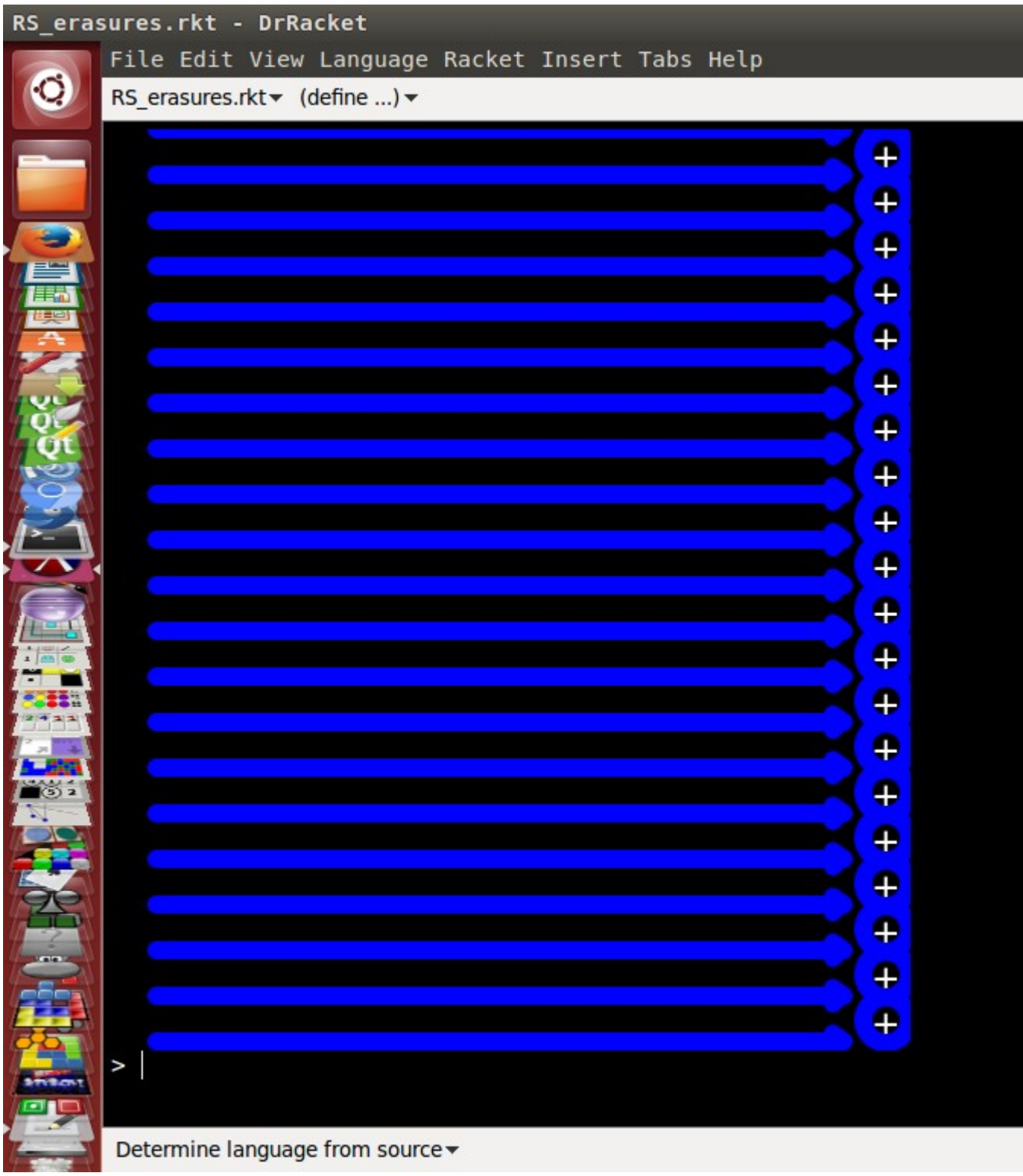
Here's usage output:

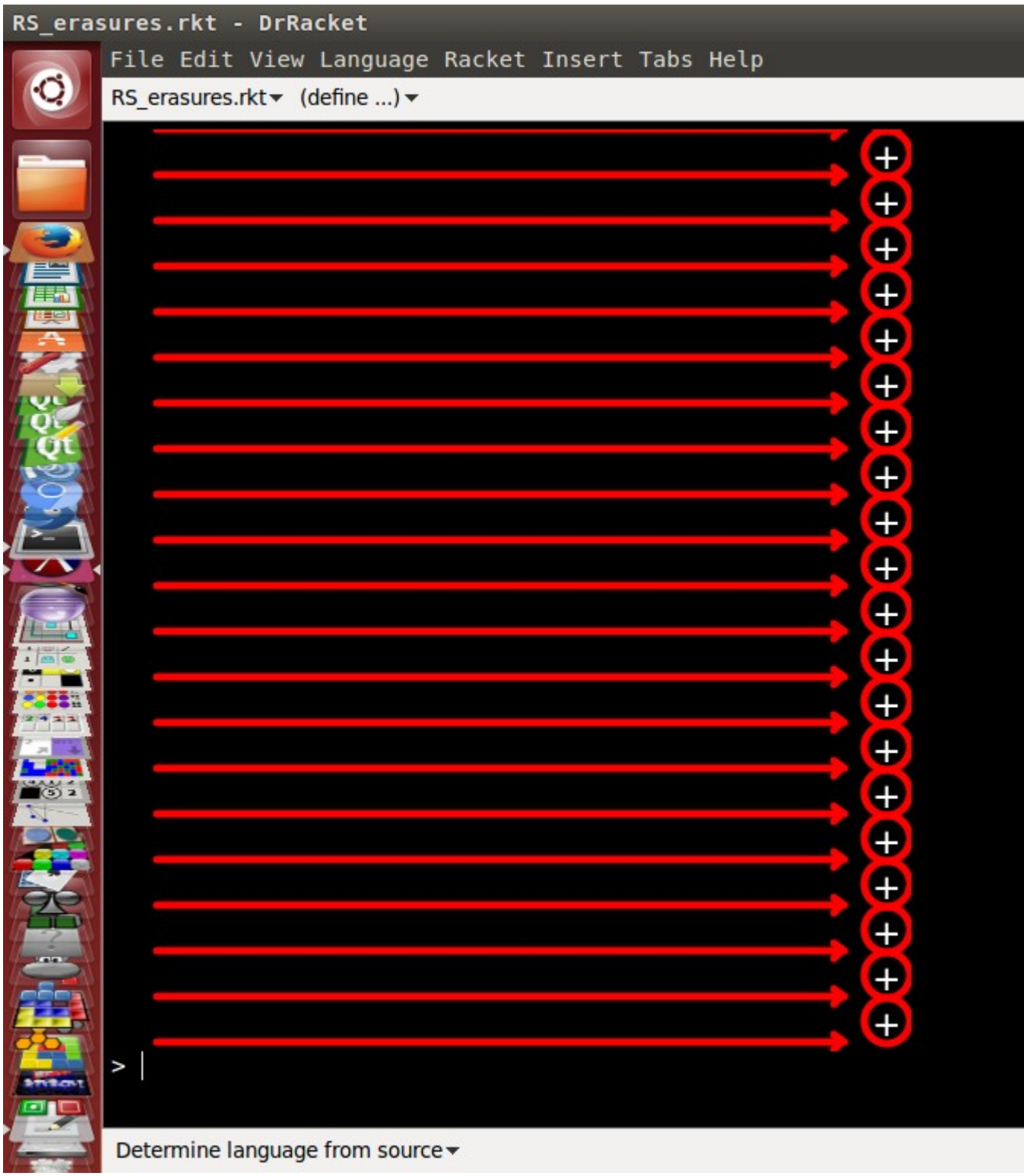

RS_erasures.rkt - DrRacket
File Edit View Language Racket Insert Tabs Help
RS_erasures.rkt (define ...)
> (list (conv-lines 50 (make-pen "red" 12 "solid" "round" "r
(conv-lines 50 (make-pen "blue" 12 "sol
(conv-lines 50 (make-pen "white" 12 "so
Determine language from source


RS_erasures.rkt - DrRacket
File Edit View Language Racket Insert Tabs Help
RS_erasures.rkt (define ...)
(list
Determine language from source


RS_erasures.rkt - DrRacket
File Edit View Language Racket Insert Tabs Help
RS_erasures.rkt (define ...)
> (conv-lines 10 (make-pen "white" 2 "solid" "round" "round"
>
Determine language from source


RS_erasures.rkt - DrRacket
File Edit View Language Racket Insert Tabs Help
RS_erasures.rkt (define ...)
>
Determine language from source


RS_erasures.rkt - DrRacket
File Edit View Language Racket Insert Tabs Help
RS_erasures.rkt (define ...)
>
Determine language from source



- Adding convolutional codes

One more post on block codes and the site would have to be renamed.

While convolutional codes too have a generator matrix, this matrix is derived from both memory elements and how these are tapped to produce the output stream. For instance in a systematic R1/2 code with memory length m=3, the interconnection can be expressed as a non-rectangular list: (() (0 1) (0 1 1)). An empty element indicates a direct connection, indicating it's a systematic code with the input bit sent straight to output at that position. A non-empty list indicates memory elements, with 0 indicating an uptapped bit and 1 indicating a bit tapped (XOR-ed) with the input and/or other memory lines to produce output.

This idea is still evolving so don't fret too much about its accuracy. But it looks like a promising way to generate impulse responses and check their validity, produce generator matrices and finally draw interconnection schematics for all convolutional code categories.

Convolutional codes will the sole focus for the next whole year.

- Non-algebraic evaluation of Reed-Solomon error values once error locations are known

This is a long post given that the problem itself is fairly complicated. Please read the whole thing.
1. The syndrome S(X) is unique for all t or fewer errors. That means, once you know the error locations, you can use the following brute force technique to arrive at the actual error values.

2. Take a look at the following functions all-comb and append-all-permutations-lists. The second one is particularly useful for non algebraic RS error value eval.

3. The function pow-pos-list-2-RS-poly converts powers positions into RS poly. There's a reverse function and please look for it in the RS PDF.

4. At the end of this post are the correct values for Golay(24, 12) generator matrix.

(all-comb '(1 2 3 4) 3)
'((2 3 4) (1 2 3) (1 3 4) (1 2 4))
> all-permutations
#
> (append-all-permutations-lists (duplicate '(11 23 44 55) 3))
'((11 11 11)
(11 11 23)
(11 11 44)
(11 11 55)
(11 23 11)
(11 23 23)
(11 23 44)
(11 23 55)
(11 44 11)
(11 44 23)
(11 44 44)
(11 44 55)
(11 55 11)
(11 55 23)
(11 55 44)
(11 55 55)
(23 11 11)
(23 11 23)
(23 11 44)
(23 11 55)
(23 23 11)
(23 23 23)
(23 23 44)
(23 23 55)
(23 44 11)
(23 44 23)
(23 44 44)
(23 44 55)
(23 55 11)

(23 55 23)
(23 55 44)
(23 55 55)
(44 11 11)
(44 11 23)
(44 11 44)
(44 11 55)
(44 23 11)
(44 23 23)
(44 23 44)
(44 23 55)
(44 44 11)
(44 44 23)
(44 44 44)
(44 44 55)
(44 55 11)
(44 55 23)
(44 55 44)
(44 55 55)
(55 11 11)
(55 11 23)
(55 11 44)
(55 11 55)
(55 23 11)
(55 23 23)
(55 23 44)
(55 23 55)
(55 44 11)
(55 44 23)
(55 44 44)
(55 44 55)
(55 55 11)
(55 55 23)
(55 55 44)

(55 55 55))
> (length (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))
64
> (length (append-all-permutations-lists2 (duplicate '(11 23 44 55) 3)))
. . BCH_RS_Prod_codes.rkt:953:20: append: contract violation
expected: list?
given: 11

> (define RS_63_57_er_vals (append-all-permutations-lists (duplicate (build-list 64 values) 3)))
> (take-right RS_63_57_er_vals 10)
'((63 63 54) (63 63 55) (63 63 56) (63 63 57) (63 63 58) (63 63 59) (63 63 60) (63 63 61) (63 63 62) (63 63 63))
> (take RS_63_57_er_vals 10)
'((0 0 0) (0 0 1) (0 0 2) (0 0 3) (0 0 4) (0 0 5) (0 0 6) (0 0 7) (0 0 8) (0 0 9))
> (length RS_63_57_er_vals)
262144
> (take (drop-right RS_63_57_er_vals 1000) 10)
'((0 0 0) (0 0 1) (0 0 2) (0 0 3) (0 0 4) (0 0 5) (0 0 6) (0 0 7) (0 0 8) (0 0 9))
> (take (drop RS_63_57_er_vals 1000) 10)
'((0 15 40) (0 15 41) (0 15 42) (0 15 43) (0 15 44) (0 15 45) (0 15 46) (0 15 47) (0 15 48) (0 15 49))

> (take (drop RS_63_57_er_locs_and_vals 1000) 10)
‘(((33 14 3) (0 15 40))
((33 14 3) (0 15 41))
((33 14 3) (0 15 42))
((33 14 3) (0 15 43))
((33 14 3) (0 15 44))
((33 14 3) (0 15 45))
((33 14 3) (0 15 46))
((33 14 3) (0 15 47))
((33 14 3) (0 15 48))
((33 14 3) (0 15 49)))
> (define RS_63_57_er_locs_and_vals_polys (map (lambda(x) (pow-pos-list-2-RS-poly x g2)) (take (drop RS_63_57_er_locs_and_vals 1000) 10)))
. . BCH_RS_Prod_codes.rkt:2649:10: take: index is too large for list
index: 33
list: ‘(-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1)
> (define RS_63_57_er_locs_and_vals_polys (map (lambda(x) (pow-pos-list-2-RS-poly x gp)) (take (drop RS_63_57_er_locs_and_vals 1000) 10)))
> gp
‘(1 0 0 0 0 1 1)
> RS_63_57_er_locs_and_vals_polys
‘((0 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1

(0 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 15 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 46 -1 -1 -1)
(0 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 15 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 47 -1 -1 -1)
(0 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 15 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 48 -1 -1 -1)
(0 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 15 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 49 -1 -1 -1))

> (define RS_63_57_er_locs_and_vals_polys_all (map (lambda(x) (pow-pos-list-2-RS-poly x gp)) RS_63_57_er_locs_and_vals))

> (define RS_63_57_er_locs_and_vals_polys__synds_all (map (lambda(x) (RS-syndromes-equation x gp num-rt))
RS_63_57_er_locs_and_vals_polys_all))

> (map (lambda(x) (RS-syndromes-equation x g2 num-rt)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))
'((18 30 6 30 24 2)
(17 13 10 13 9 28)
(14 23 3 23 15 12)
(29 22 0 22 11 21)
(26 9 1 24 17 15)
(29 24 14 9 18 22)
(25 22 17 15 1 24)
(17 23 25 11 26 13)
(8 0 16 19 11 18)
(19 -1 7 8 23 17)
(-1 19 27 0 13 14)
(5 6 20 3 24 29)
(13 6 25 0 1 26)
(30 10 4 -1 14 29)
(11 3 26 19 17 25)
(15 0 1 6 25 17)
(22 22 3 15 16 -1)
(15 15 5 22 7 0)
(9 9 6 24 27 8)
(30 30 19 13 20 10)
(13 23 17 23 3 1)
(30 11 18 11 5 14)
(11 30 1 30 6 17)
(15 9 26 9 19 25)
(2 6 27 5 20 9)
(28 10 21 3 12 24)
(12 3 16 10 28 22)
(21 0 28 8 16 23)
(26 0 26 10 6 11)
(29 -1 29 6 10 23)
(25 19 25 5 3 13)
(17 6 17 -1 0 24)

(15 7 12 21 1 21)
(22 16 20 27 14 27)
(24 21 2 7 17 7)
(13 12 7 2 25 2)
(30 2 23 20 15 13)
(13 28 11 12 22 30)
(23 12 30 28 24 11)
(22 21 9 16 13 15)
(28 17 -1 17 25 29)
(2 18 0 18 4 26)
(20 1 8 1 26 4)
(27 26 10 26 1 18)
(29 26 9 1 24 17)
(26 29 24 14 9 18)
(4 25 22 17 15 1)
(18 17 23 25 11 26)
(12 26 26 17 20 8)
(20 29 29 18 12 19)
(2 25 25 1 28 -1)
(7 17 17 26 16 5)
(16 1 19 4 19 17)
(7 14 8 25 8 18)
(27 17 0 29 0 1)
(20 25 3 14 3 26)
(9 12 13 21 16 22)
(24 20 30 27 7 15)
(22 2 11 7 27 9)
(23 7 15 2 20 30)
(6 7 3 7 0 13)
(10 16 5 16 -1 30)
(3 21 6 21 19 11)
(0 12 19 12 6 15))

> (map (lambda(x) (RS-poly-syndromes x g2 num-rt)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))
'(((0 0 1 0 0) (1 1 1 0 0) (1 0 1 0 0) (1 0 0 1 0) (1 0 1 0 0) (1 1 0 0 1))
((0 0 1 0 1) (1 1 1 0 1) (1 0 1 0 1) (1 0 0 1 1) (1 0 1 0 1) (1 1 0 0 0))
((1 1 1 1 0) (0 0 1 1 0) (0 1 1 1 0) (0 1 0 0 0) (0 1 1 1 0) (0 0 0 1 1))
((1 0 1 1 1) (0 1 1 1 1) (0 0 1 1 1) (0 0 0 0 1) (0 0 1 1 1) (0 1 0 1 0))
((0 0 1 1 0) (1 1 0 0 0) (1 1 1 0 0) (0 0 0 1 0) (1 1 1 0 1) (0 1 0 1 1))
((0 0 1 1 1) (1 1 0 0 1) (1 1 1 0 1) (0 0 0 1 1) (1 1 1 0 0) (0 1 0 1 0))
((1 1 1 0 0) (0 0 0 1 0) (0 0 1 1 0) (1 1 0 0 0) (0 0 1 1 1) (1 0 0 0 1))
((1 0 1 0 1) (0 1 0 1 1) (0 1 1 1 1) (1 0 0 0 1) (0 1 1 1 0) (1 1 0 0 0))
((1 1 0 0 1) (0 1 1 1 1) (1 1 0 1 1) (0 1 1 0 0) (0 0 0 0 1) (1 1 0 1 0))
((1 1 0 0 0) (0 1 1 1 0) (1 1 0 1 0) (0 1 1 0 1) (0 0 0 0 0) (1 1 0 1 1))
((0 0 0 1 1) (1 0 1 0 1) (0 0 0 0 1) (1 0 1 1 0) (1 1 0 1 1) (0 0 0 0 0))
((0 1 0 1 0) (1 1 1 0 0) (0 1 0 0 0) (1 1 1 1 1) (1 0 0 1 0) (0 1 0 0 1))
((0 1 0 1 1) (0 0 0 1 0) (0 0 0 0 1) (1 0 0 0 1) (1 0 0 1 0) (1 0 1 0 1))
((0 1 0 1 0) (0 0 0 1 1) (0 0 0 0 0) (1 0 0 0 0) (1 0 0 1 1) (1 0 1 0 0))
((1 0 0 0 1) (1 1 0 0 0) (1 1 0 1 1) (0 1 0 1 1) (0 1 0 0 0) (0 1 1 1 1))
((1 1 0 0 0) (1 0 0 0 1) (1 0 0 1 0) (0 0 0 1 0) (0 0 0 0 1) (0 0 1 1 0))
((0 0 0 0 0) (0 1 1 0 0) (0 0 1 1 0) (0 1 0 0 0) (0 0 1 1 1) (0 0 1 1 1))
((0 0 0 0 1) (0 1 1 0 1) (0 0 1 1 1) (0 1 0 0 1) (0 0 1 1 0) (0 0 1 1 0))
((1 1 0 1 0) (1 0 1 1 0) (1 1 1 0 0) (1 0 0 1 0) (1 1 1 0 1) (1 1 1 0 1))
((1 0 0 1 1) (1 1 1 1 1) (1 0 1 0 1) (1 1 0 1 1) (1 0 1 0 0) (1 0 1 0 0))
((0 0 0 1 0) (0 1 0 0 0) (0 1 1 1 0) (1 1 0 0 0) (0 1 1 1 0) (1 0 1 0 1))
((0 0 0 1 1) (0 1 0 0 1) (0 1 1 1 1) (1 1 0 0 1) (0 1 1 1 1) (1 0 1 0 0))
((1 1 0 0 0) (1 0 0 1 0) (1 0 1 0 0) (0 0 0 1 0) (1 0 1 0 0) (0 1 1 1 1))
((1 0 0 0 1) (1 1 0 1 1) (1 1 1 0 1) (0 1 0 1 1) (1 1 1 0 1) (0 0 1 1 0))
((1 1 1 0 1) (1 1 1 1 1) (0 1 0 0 1) (1 0 1 1 0) (1 0 0 1 0) (0 0 1 0 0))
((1 1 1 0 0) (1 1 1 1 0) (0 1 0 0 0) (1 0 1 1 1) (1 0 0 1 1) (0 0 1 0 1))
((0 0 1 1 1) (0 0 1 0 1) (1 0 0 1 1) (0 1 1 0 0) (0 1 0 0 0) (1 1 1 1 0))
((0 1 1 1 0) (0 1 1 0 0) (1 1 0 1 0) (0 0 1 0 1) (0 0 0 0 1) (1 0 1 1 1))
((0 1 1 1 1) (1 0 0 1 0) (1 0 0 1 1) (0 1 0 1 1) (0 0 0 0 1) (0 1 0 1 1))
((0 1 1 1 0) (1 0 0 1 1) (1 0 0 1 0) (0 1 0 1 0) (0 0 0 0 0) (0 1 0 1 0))
((1 0 1 0 1) (0 1 0 0 0) (0 1 0 0 1) (1 0 0 0 1) (1 1 0 1 1) (1 0 0 0 1))
((1 1 1 0 0) (0 0 0 0 1) (0 0 0 0 0) (1 1 0 0 0) (1 0 0 1 0) (1 1 0 0 0))

((1 0 1 1 1) (0 0 0 1 0) (1 0 1 1 1) (1 1 1 1 0) (0 1 1 0 1) (0 0 1 1 0))
((1 0 1 1 0) (0 0 0 1 1) (1 0 1 1 0) (1 1 1 1 1) (0 1 1 0 0) (0 0 1 1 1))
((0 1 1 0 1) (1 1 0 0 0) (0 1 1 0 1) (0 0 1 0 0) (1 0 1 1 1) (1 1 1 0 0))
((0 0 1 0 0) (1 0 0 0 1) (0 0 1 0 0) (0 1 1 0 1) (1 1 1 1 0) (1 0 1 0 1))
((1 0 1 0 1) (0 0 1 1 0) (1 1 1 1 1) (0 1 1 1 0) (0 0 1 0 0) (1 0 1 0 0))
((1 0 1 0 0) (0 0 1 1 1) (1 1 1 1 0) (0 1 1 1 1) (0 0 1 0 1) (1 0 1 0 1))
((0 1 1 1 1) (1 1 1 0 0) (0 0 1 0 1) (1 0 1 0 0) (1 1 1 1 0) (0 1 1 1 0))
((0 0 1 1 0) (1 0 1 0 1) (0 1 1 0 0) (1 1 1 0 1) (1 0 1 1 1) (0 0 1 1 1))
((0 1 0 1 0) (1 0 0 0 1) (1 1 0 0 0) (0 0 0 0 0) (1 1 0 0 0) (0 0 1 0 1))
((0 1 0 1 1) (1 0 0 0 0) (1 1 0 0 1) (0 0 0 0 1) (1 1 0 0 1) (0 0 1 0 0))
((1 0 0 0 0) (0 1 0 1 1) (0 0 0 1 0) (1 1 0 1 0) (0 0 0 1 0) (1 1 1 1 1))
((1 1 0 0 1) (0 0 0 1 0) (0 1 0 1 1) (1 0 0 1 1) (0 1 0 1 1) (1 0 1 1 0))
((1 1 0 0 0) (1 1 1 0 0) (0 0 0 1 0) (1 1 1 0 1) (0 1 0 1 1) (0 1 0 1 0))
((1 1 0 0 1) (1 1 1 0 1) (0 0 0 1 1) (1 1 1 0 0) (0 1 0 1 0) (0 1 0 1 1))
((0 0 0 1 0) (0 0 1 1 0) (1 1 0 0 0) (0 0 1 1 1) (1 0 0 0 1) (1 0 0 0 0))
((0 1 0 1 1) (0 1 1 1 1) (1 0 0 0 1) (0 1 1 1 0) (1 1 0 0 0) (1 1 0 0 1))
((1 1 0 1 0) (1 1 1 1 1) (1 1 0 0 0) (0 1 0 1 1) (0 1 0 1 1) (1 1 1 1 0))
((1 1 0 1 1) (1 1 1 1 0) (1 1 0 0 1) (0 1 0 1 0) (0 1 0 1 0) (1 1 1 1 1))
((0 0 0 0 0) (0 0 1 0 1) (0 0 0 1 0) (1 0 0 0 1) (1 0 0 0 1) (0 0 1 0 0))
((0 1 0 0 1) (0 1 1 0 0) (0 1 0 1 1) (1 1 0 0 0) (1 1 0 0 0) (0 1 1 0 1))
((1 1 0 0 0) (1 1 0 1 1) (1 0 0 0 0) (1 1 0 1 1) (0 0 0 1 0) (0 1 1 0 0))
((1 1 0 0 1) (1 1 0 1 0) (1 0 0 0 1) (1 1 0 1 0) (0 0 0 1 1) (0 1 1 0 1))
((0 0 0 1 0) (0 0 0 0 1) (0 1 0 1 0) (0 0 0 0 1) (1 1 0 0 0) (1 0 1 1 0))
((0 1 0 1 1) (0 1 0 0 0) (0 0 0 1 1) (0 1 0 0 0) (1 0 0 0 1) (1 1 1 1 1))
((0 0 1 1 1) (0 1 1 0 0) (1 0 1 1 1) (1 0 1 0 1) (1 1 1 1 0) (1 1 1 0 1))
((0 0 1 1 0) (0 1 1 0 1) (1 0 1 1 0) (1 0 1 0 0) (1 1 1 1 1) (1 1 1 0 0))
((1 1 1 0 1) (1 0 1 1 0) (0 1 1 0 1) (0 1 1 1 1) (0 0 1 0 0) (0 0 1 1 1))
((1 0 1 0 0) (1 1 1 1 1) (0 0 1 0 0) (0 0 1 1 0) (0 1 1 0 1) (0 1 1 1 0))
((1 0 1 0 1) (0 0 0 0 1) (0 1 1 0 1) (0 1 0 0 0) (0 1 1 0 1) (1 0 0 1 0))
((1 0 1 0 0) (0 0 0 0 0) (0 1 1 0 0) (0 1 0 0 1) (0 1 1 0 0) (1 0 0 1 1))
((0 1 1 1 1) (1 1 0 1 1) (1 0 1 1 1) (1 0 0 1 0) (1 0 1 1 1) (0 1 0 0 0))
((0 0 1 1 0) (1 0 0 1 0) (1 1 1 1 0) (1 1 0 1 1) (1 1 1 1 0) (0 0 0 0 1)))
> (length (map (lambda(x) (RS-poly-syndromes x g2 num-rt)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3))))

64
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x g2 num-rt)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))))
64
> g2
'(1 0 1 0 0 1)

> gp
'(1 0 0 0 0 1 1)

> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x g1 num-rt)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))))
64
> g1
'(1 0 0 1 1)
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x g1 10)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))))
64
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x gp 12)) (append-all-permutations-lists (duplicate '(11 23 44 55) 3)))))
64
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x gp 12)) (append-all-permutations-lists (duplicate '(1 2 4 5) 3)))))
64
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x g1 12)) (append-all-permutations-lists (duplicate '(1 2 4 5) 3)))))
64
> (length (remove-duplicates (map (lambda(x) (RS-poly-syndromes x g2 12)) (append-all-permutations-lists (duplicate '(1 2 4 5) 3)))))

CORRECT VALUES FOR GOLAY GENERATOR MATRIX

```
> golay-gm
‘((1 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 1 1 1 0 1 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0)
(0 0 1 1 1 0 1 0 1 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0)
(0 1 1 1 0 1 0 1 1 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0)
(1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0)
(1 1 0 1 0 1 1 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0)
(1 0 1 0 1 1 0 0 0 1 1 1 0 0 0 0 0 0 1 0 0 0 0 0)
(0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0)
(1 0 1 1 0 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0)
(0 1 1 0 0 0 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 1 0 0)
(1 1 0 0 0 1 1 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1))

> (define gol_24_12 (encode-linear-code golay-gm))
> (check-cyclicity gol_24_12)
#f

> (define gol_24_12_w (sort-by-degree gol_24_12))
> (car gol_24_12_w)
‘(1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1)
```

> (weight-enum gol_24_12)
‘(1 0 0 0 0 0 66 0 363 0 990 0 1256 0 990 0 363 0 66 0 0 0 0 0 1)
> (min-dist-gm golay-gm)
6
> (min-dist2 gol_24_12)
6
> (take gol_24_12 10)
‘((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1)
(1 1 0 0 0 1 1 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 1 1 1 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1)
(0 1 1 0 0 0 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0 1 0 0)
(1 0 0 1 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1)
(1 0 1 0 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0)
(0 1 0 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1)
(1 0 1 1 0 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0)
(0 1 0 0 1 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 1 0 0 1))

- Conclusive proof Racket curbs your material needs exponentially

This software was written entirely on a 2009 model Dell Vostro 1510 which I still use. The machine has aged well except for the scratched up display.
Clearly Racket has some unknown magical properties that have ensured the laptop’s longevity.

That’s just the beginning. On average some 500 hours of coding effort per year, starting in Feb 2009, amounts to several thousand hours of cheapest and most fulfilling entertainment on the planet.

It will work for you if you’re looking to solve a long-tern problem and have the time and patience to see it through to a reasonably good solution.

For me the starting point of real challenge was Reed-Muller. The Lin-Costello book details the parity checksums for RM(2, 4) only. It took months to find a generalization.

Maybe you can find something like that. Who knows.

Next Page→

Indice ––––––––––
Blog at WordPress.com.

You can’t learn Racket from these YouTube videos. Instead study the eye-popping, jaw-dropping, awe-inspiring official Racket documentation.

In particular, print out the first 10 chapters of the latest Racket Guide PDF.

What follows is at least months of gruel. When you can animate the binary search in Racket Guide, insert nodes, find nodes, rebuild the tree with a different element in the list of structures… you have arrived. Almost.

Then find the chapter on Lin-Costello that details the Golay decoder. By now you'd be so proud that you would've put up your version of the animated search tree somewhere, so I'll find your site and mail you the function outline to build the Golay decoder.

I'm kidding. You'll have to come up with that yourself.

- The test code is woefully out of date with the functions

The test polynomials p1, p2 etc were put in long before I implemented the error value evaluators. It's best to build completely new test cases. Steps for building RS test cases:

1. Write a new function that generates a valid RS test vector, then corrupt it with a known error pattern. See below:

(define (RS-eucl-dec-test-rand-corrtd-poly g t)
(let* ([rs-p (encode-random-RS-poly g (* (add1 t) 2))][er-p (random-n-symb-RS-error-pattern g t)][p (add-RS-polys er-p rs-p g)]

You need to combine these 3 steps (encoding random RS poly, generating a random RS error vector, then adding this to the clean vector) and give it a name like encode-random-corr-RS-vector.

2. An alternative to this is to build a full fledged bank of test vectors taken from an image perhaps. Use the Racket logo image if you want entertainment.

3. If you follow the above (second) approach, noise vector syndromes can be displayed as a grid with HSV or RGB components that correspond to the syndromes of alpha^1, alpha^2, alpha^3. For clean uncorrupted vectors, use median 128 for HSV or RGB scale, then add the syndromes, then do a modulo of n where n is the code word length such as 15, 31, 63, 127 or 255.

4. If do not want to use Racket for test vector generation, dump REPL out as binary file. Then write a bitstream parser in a GUI rich IDE like Qt or GtkSharp. Actually I'm thinking of doing this soon using the Drawing example in Qt.

- Function contracts will take effect only if you use the misc. test code REPL

That means it's best to save all 3 source files and run the test code file even if you want to use only the Hamming code functions, for instance.
Another advantage of this approach is you can add any new functions to the misc test code file without modifying the other two files. The Golay(24.12) decoder is missing and it's a great intermediate level exercise to add to the test code file.

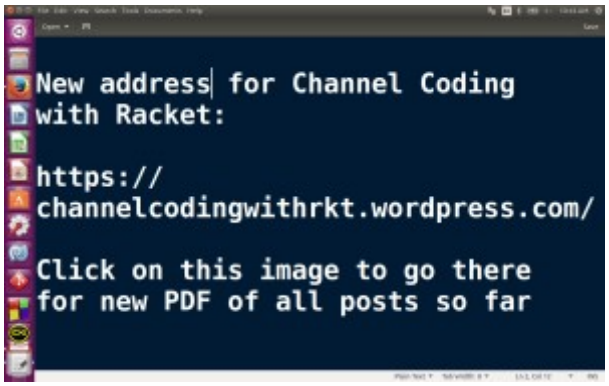
When function contracts don't take effect at the suite level, some core language library will flag an exception and it might be more difficult to trace. So, please use only the misc test code REPL.

Another slightly less difficult exercise is to generate all RS codewords at a given symbol position.

- New PDF of all posts so far

All links in this PDF point to the new site address:

All_posts_21_May_2017



- Bugs found in eval-RS-poly-root-list-2 function

In the body of the above mentioned function:
(car (divide-RS-polys (car (divide-RS-polys p (list (last p)) g num-rt)) (list x 0) g num-rt))

1. num-rt is a global variable set to 6. No wonder the evaluators aren't working. The function signature needs to be changed to accept num-r as a parameter. All callers of eval-RS-poly-root-list-2 will in turn have to take in num-r, right upto eucl-dec-end-2-end-test. More on this below.

2. The inner 'car' factors out the constant term, which is the last element of the list, from the derivative of Sigma(X), that's Sigma'(X):
(car (divide-RS-polys p (list (last p)) g num-rt))

But it does not multiply this factored out term to Sigma'(X), which means the evaluated error values do not get DIVIDED by this factored out term. This is the second bug.

The solution is very, very simple. Rewrite the following functions entirely after modifying eval-RS-poly-root-list-2 suitably to 1. Take num-r as a parameter and 2. Return a list consisting of the factored out term and the present return value:

1. eval-RS-poly-fraction-root-list

2. eval-RS-poly-deriv-fraction-root-list

3. eval-RS-poly-fraction2-root-list

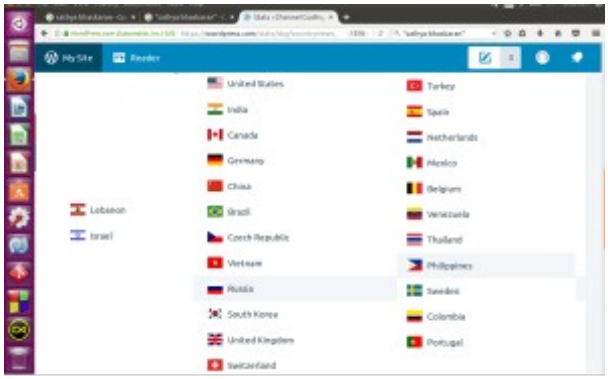
4. eval-RS-poly-deriv-fractions-root-list

5. eval-RS-poly-fractions2-root-list

This is why I told you that implementing Frequency Domain Decoding of error values is a much better idea.

- The enduring international appeal of our fascinating subject – site stats

Apparently people from the following 25 countries do not mind appalling documentation if there's enough functionality:



- Use RS-decode-eucl for finding the error locator polynomial only

Then use some other technique to find out the error values. I'm going to post in detail on Reed-Solomon error value evaluators soon, but the most important part here is how to interpret a derivative of Sigma(x). My interpretation is that only the coefficients corresponding to odd-number powers of X will be present in the derivate because multiplying a coefficient by an even number is equivalent to adding it to itself, which results in a numerical 0 (or a -1 in the RS poly representation).
That brings us to an ambiguity in using printable_poly for RS polys. The old post on the function need not be updated if you interpret all coefficients as powers of primitive element.

- Clojure port canceled. No port of ANY existing codes.

Don't you wonder, why have this suite in Racket? Why not just use the free Matlab version for Linux?
The answer is that you want your favorite domain's code base in your favorite language. Turns out Racket is the best suited for coding theory as a whole. That includes source coding that this suite does not even cover.

Matlab is very useful for beginners but surprisingly weak for serious researchers. Matlab's communications toolbox has vast scope, but how many new channel codes have been discovered through it?

Just because a language has a large user base doesn't mean it's enjoyable to write code in it. Had it not been for the sheer joy of coding in Racket, I would've stopped with Reed-Muller. Binary BCH took a whole year to code up. Reed-Solomon took another two years.

That means you'll have to learn Racket to use this suite. There are at least 15-20 advanced problems to be added to this suite in the future, including squaring construction of codes, trellises for linear block codes, hierarchical decoding of Reed-Muller codes, erasure correction, entertaining ways to generate test vectors, several rounds of optimizations, techniques for generating candidate vectors for soft-decision decoding, codeword generation for large-block length codes generation, generalizing and expanding the scope of the binary search tree to visualize candidate vectors, and we haven't even covered games yet.

Some of these future features might be in Rust, which seems like an equally delightful option.

- It will be codedchannel.wordpress.com soon

The domain codedchannel.com expires in May. This site will become a free wp blog.

- Drawing Reed-Solomon polynomials with 2htdp/image

1. You'll need to tweak the following snippets to suit your DrRacket background color setting.
2. Only alpha and beta are provided. Sigma is much easier to draw and it's left as an exercise. You need to scale these symbols with the scale/xy function in 2htdp/image package.

3. The main RS error value evaluator featured in Lin-Costello textbook isn't able to resolve more than 2 errors. The second evaluator mentioned in the book carries no worked out examples, and my interpretation of it is probably wrong because it isn't resolving a SINGLE error.

4. The best way out of this seems to be to implement Frequency Domain Decoding of error values. If I ever implement this, I won't be posting the whole code here, so if you're interested you'll anyway have to build this yourself.

Now to the actual code snippets:

1. Alpha symbol:

(rotate -90 (add-curve (add-curve (rectangle 100 100 "solid" "black")
81 20 180 1/10
50 80 0 1 (make-pen "white" 11 "solid" "round" "round")) 19 20 0 1/10
50 80 180 1 (make-pen "white" 11 "solid" "round" "round")))

2. Beta symbol:

(add-line (beside (rectangle 30 140 "solid" "black") (crop 40 0 140 140
(add-curve (add-curve (rectangle 40 100 "solid" "black") 20 15 180 1 20
120 180 1 (make-pen "white" 8 "solid" "round" "round")) 20 15 0 1 20
120 0 1 (make-pen "white" 8 "solid" "round" "round")))) 30 15 30 150
(make-pen "white" 8 "solid" "round" "round"))

3. Outline for recursing through a Reed-Solomon poly using these symbols:

(let rec([x (reverse (drop-right p-l 1))][i 1][acc (if (= -1 (last p-l))
(rectangle 20 20 "solid" "black") ... (if (null? x) (scale/xy 1/3 1/3 acc) (if (=
-1 (car x)) (rec (cdr x) (add1 i) acc)
(rec (cdr x) (add1 i) (beside (beside (ele-pow ele (car x) i) (text "+" 120
"white")) acc)))))

Good luck!

←Previous PageNext Page→

Indice ––––––––––
Blog at WordPress.com.

Channel Coding with Racket

- About
- Contact
- Miscellaneous test code
- source: Binary BCH, Reed-Solomon, Golay, Product codes
- Source: Hamming, Reed-Muller, Cyclic, and general linear block code properties
- Things to do
- Update
- Usage reference

Channel coding suite

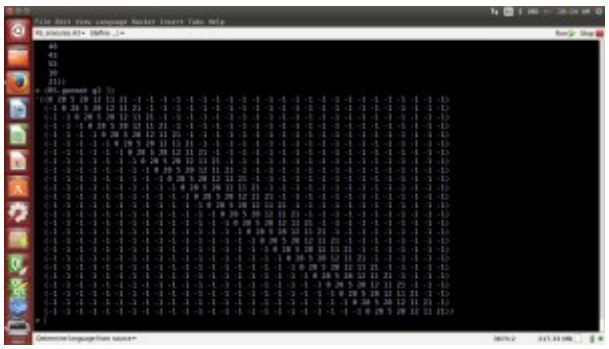
Tumblr ↗️

- Viewing full-res versions of images in posts

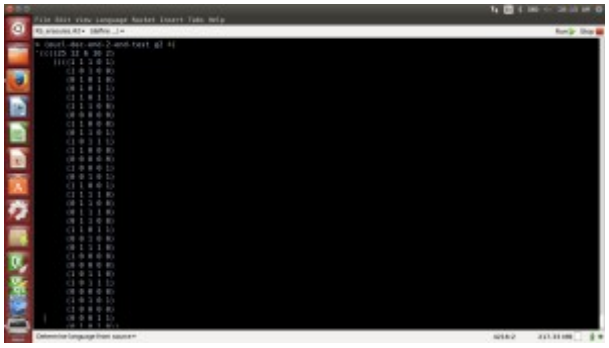
How to view full-res versions of images posted here
1. Click ‘View Image’ on any image on a post.
2. In the URL of the displayed image, delete the portion after the file name extension. For instance in the ‘More Progress ’ post below, remove the trailing ?w=450&h=254 part after the .png extension.
3. Hit enter.
4. The original, full resolution image will load now.

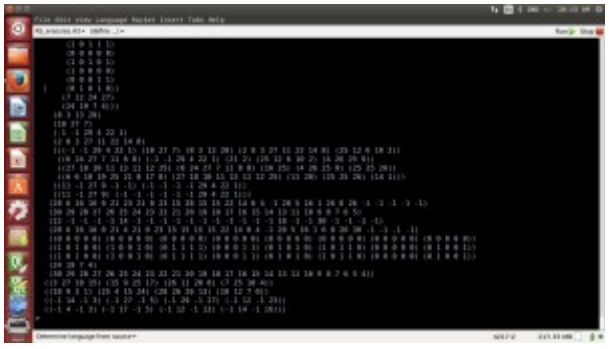
Here’s a repeat of images posted in the usage reference page:

1. Reed-Solomon generator matrix:

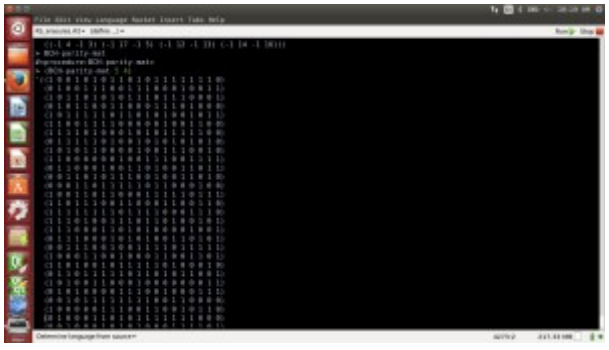


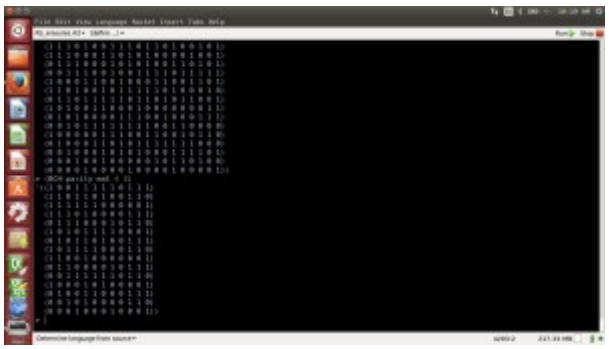
2. Reed-Solomon Euclidean decoder end to end test (please read previous post for interpreting its output)





3. Binary BCH parity matrix (to check the validity and accuracy of the corresponding Binary BCH generator matrix):





4. All Binary polynomials are MSB first and Reed-Solomon polynomials are most significant symbol first:



- Interpreting the Reed-Solomon Euclidean decoder output

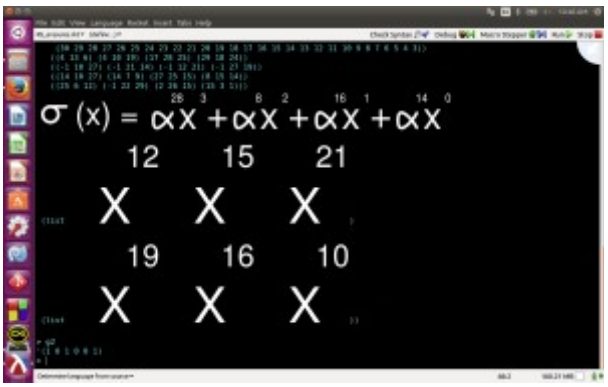
First, here is the augmented output function – whose code I'll paste here soon – of the Euclidean Reed-Solomon decoder for a random but valid RS codeword vector corrupted by a random error vector supplied by the last argument:

```
> (eucl-e2e-vec-erloc g2 3) ; <— WILL SUPPLY CODE FOR THIS
FUNCTION SOON
(list
'((((28 8 16 14)
((((0 1 0 0 1)
(0 1 1 0 0)
(0 1 0 0 1)
(0 1 1 0 1)
(1 0 0 1 1)
(1 1 1 0 1)
(1 0 0 1 1)
(1 0 1 0 0)
(0 0 1 0 1)
(1 1 0 1 1)
(1 0 1 0 1)
(0 0 0 0 0)
(0 1 0 0 0)
(0 0 1 0 0)
(0 0 0 0 0)
(1 1 1 1 1)
(1 0 0 0 0)
(0 0 0 0 1)
(1 1 0 1 1)
(1 1 0 0 1)
(0 0 0 0 0)
(1 1 0 1 1)
(0 0 1 1 1)
(0 0 0 1 0)
(1 0 1 0 1)
(1 0 0 1 1)
(1 0 0 0 0)
(0 1 0 0 1)
(1 0 1 1 1)
(0 0 1 1 0)
```

(1 0 0 0 0))
(12 15 21)
(19 16 10)))
(8 10 10 4)
(29 18)
(-1 -1 26 21 4)
(5 0 3 21 10 7 28)
(((-1 -1 26 21 4) (29 18) (8 10 10 4) (5 0 3 21 10 7 28) (28 8 16 14))
((3 14 27 27 17 14) (-1 -1 26 21 4) (20 25) (28 8 16 14) (8 23 24))
((23 7 27 19 23 4 29) (3 14 27 27 17 14) (8 1) (8 23 24) (0 8)))
((28 10 29 -1 -1) (-1 -1 -1 26 21 4))
((28 10 29) (-1 -1 -1 -1 -1 26 21 4)))
(29 4 25 2 15 11 12 0 27 26 8 25 14 10 -1 23 27 29 3 19 5 17 11 14 22 23 29 12 -1 -1 -1)
(30 29 28 27 26 25 24 23 22 21 20 19 18 17 15 14 13 12 11 10 9 8 7 6 5 4 3)
(26 -1 -1 -1 11 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 23 -1 -1 -1 -1 -1 -1 -1)
(29 4 25 2 15 11 12 0 27 26 8 20 14 10 26 23 27 29 3 19 15 17 11 14 22 23 29 12 -1 -1 -1)
((0 0 0 0 0) (0 0 0 0 0) (0 0 0 0 0) (0 0 0 0 0) (0 0 0 0 0) (0 0 0 0 0))
((1 0 1 1 1) (0 0 0 1 1) (1 0 1 0 0) (0 1 0 0 0) (1 0 0 0 0) (0 0 0 1 0))
((1 1 0 1 1) (0 0 0 0 0) (0 1 1 1 1) (1 1 1 1 1) (0 0 0 0 0) (1 1 1 0 1))
(19 16 10)
(30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3))
((4 13 6) (4 10 19) (17 28 25) (29 18 24))
((-1 10 27) (-1 21 14) (-1 12 21) (-1 27 19))
((14 10 27) (14 7 9) (27 25 15) (8 15 14))
((25 6 12) (-1 22 29) (2 26 15) (15 3 1)))

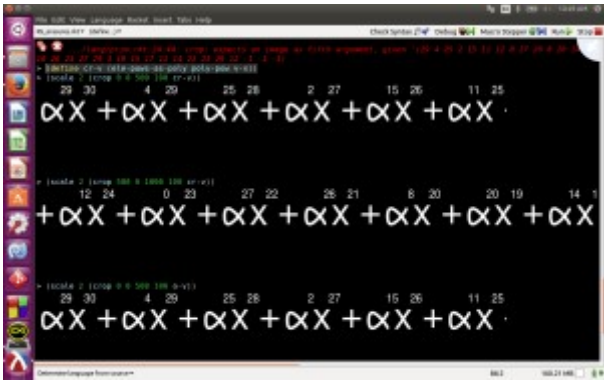
And this is how to interpret the above output:

1. Sigma (x), inverses of error locations and the error locations themselves:

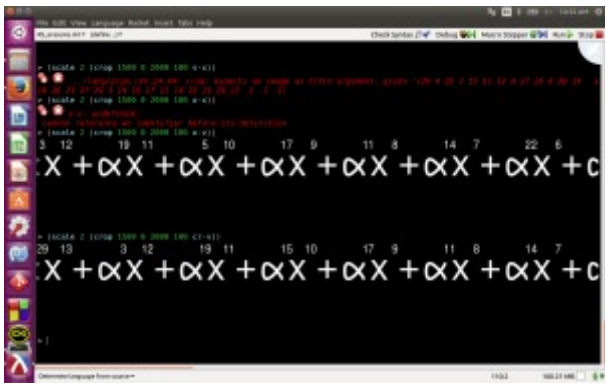


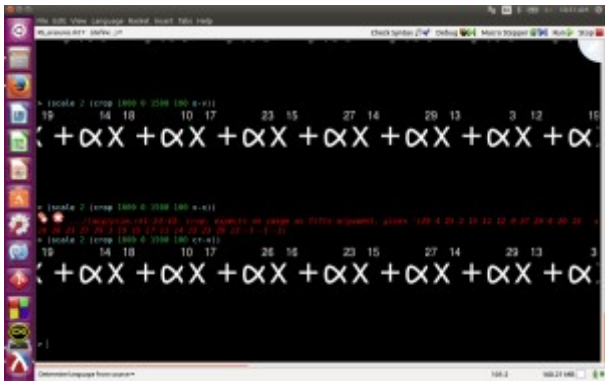
2. The original valid RS codeword and the corrupted codeword:





3. You might have noticed the error locations: 10, 16, and 19:







On the next post I'll write about the error value evaluators and supply the code for displaying RS polynomials as seen in the images above.

- It's not a bug. It's not a feature. It's an exercise problem.

There is no better way to learn this suite other than through trial and awesome.
To this end it seems best to introduce half-solved problems and let users complete the rest.

If you look at the miscellaneous test code PDF on top, the RS-decode-berl function is incomplete. It's basically a copy-paste in haste of the RS-decode-eucl function in one of the other PDF's. Now RS-decode-eucl is complete as far as error locator polynomial is concerned. But the error value evaluators are not able to locate more than 2 errors.

In the Lin-Costello textbook the Berlekamp example corrects 3 errors. So this must be a better algorithm. All the support functions such as adding RS polynomials, multiplying RS polynomials, even division etc is already available. Someone could take it up.

Another equally long exercise would be the Finite Field Fourier Transform and erasure correction using it. This is another Lin-Costello textbook example.

Please write to me if you're willing to attempt these. My mail id is in the About page.

- Demonstrating decodability of codes is the main goal here

. . .and NOT building industry-strength decoders for specific codes belonging to any particular category of codes. Commercial decoders are protected by a big wall of patents to the extent that the original textbook algorithm is unrecognizable. We'll get to that in a later post. But first let's reflect how a coding theory student approaches the whole subject.
It takes one long enough to understand the Hamming code basics, let alone its subtleties. Even if a student hasn't given up on coding theory by now, there's the very real doubt that there can anything beyond Hamming. Reed-Muller is at least MLD, so it's close enough.

But Reed-Solomon is a category that you won't be convinced even works until you see for yourself that Euclidean algorithm does locate the errors. This kind of eureka moment is what this suite aims to address.

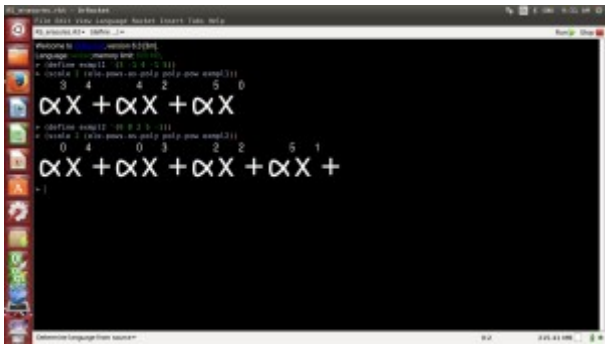
With Reed-Solomon, then there's the various methods to evaluate the actual error values. There are two error evaluators implemented in this suite. Frequency domain decoding of error values is the third technique that maybe someone can add in the future.

- Recap: Reed-Solomon codewords

This is from an earlier post about how this suite represents RS polys: Example 1: '(3 -1 4 -1 5) stands for (alpha^3 * X^4) + (alpha^4 * X^2) + alpha^5. Missing coefficient powers of X are represented as -1's. The power of X is the position of the element in the list , with the most significant power first.

Example 2: '(0 0 2 5 -1) stands for X^4 + X^3 + (alpha^2 * X^2) + (alpha^5 * X). Any 0 value in the list indicates alpha^0 (= 1) at that position in the list.

Here's an image of the above 2 example polys in graphical form (excuse the trailing '+' sign in example 2 – this is a minor bug):



- Accuracy of Binary BCH generator matrices

Here are two ways to check the accuracy of Binary BCH generator matrix using arbitrary generator polynomial:
1. Compute the minimum distance of the code using the generator matrix by invoking the min-dist-gm function. Please read the following example carefully:

> (define ygm '((1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0)
(0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0)
(0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0)
(0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0)
(0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0)
(0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0)
(0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0)
(0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0)
(0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0)
(0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1)))

Now compute the minimum distance using the correct function for this occasion:

> (min-dist-gm ygm)
11

THE FOLLOWING FUNCTION IS THE WRONG FUNCTION TO USE (because it applies to all codewords, not gen mats):
> (min-dist ygm)
12

Predictably, the correct minimum distance function takes longer:
> (time (min-dist-gm ygm))
cpu time: 237 real time: 236 gc time: 60
11

And the wrong minimum distance function takes no time to return the wrong result:
> (time (min-dist ygm))
cpu time: 0 real time: 1 gc time: 0
12
>

2. The second way to check BCH gen mat accuracy is to use the check-cyclicity-gm function:

> (check-cyclicity-gm ygm)
#t

Evidently it takes its sweet time to compute the cyclicity:

> (time (check-cyclicity-gm ygm))
cpu time: 15185 real time: 15178 gc time: 77
#t

That took FIFTEEN SECONDS to tell you that the codeword set generated using ygm was indeed cyclic.

Now a look at the INCORRECT function for checking the cyclicity using generator matrices:

> (check-cyclicity ygm)
#f

Fortunately it returns FALSE. So no need to check its performance.

- Reed-Solomon generator polynomials

For the generator polynomial X^6 + X + 1, here is an image of different RS generator polynomials for 2, 3, 4, 5 and 6 errors with the following command:
> gp
‘(1 0 0 0 0 1 1)

> (map (lambda (x) (RS-poly x gp)) ‘(4 6 8 10 12))

OUTPUT:

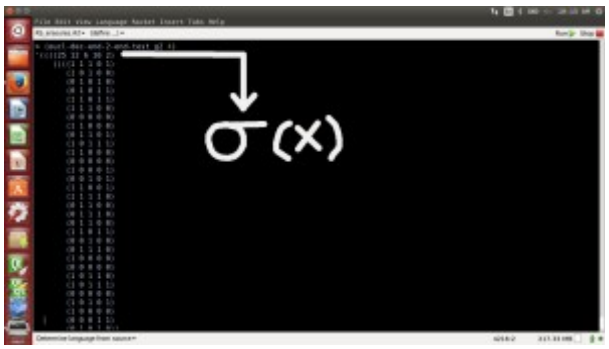
‘((0 19 41 24 10) <—- 2-error correcting GP with 4 roots
(0 59 48 43 55 10 21) <—- 3-error correcting GP with 6 roots
(0 43 59 31 10 40 14 7 36) <—– 8 roots, 4 errors
(0 56 28 47 53 60 1 6 61 37 55) <—– 10 roots, 5 errors
(0 60 12 44 6 60 28 10 32 20 1 62 15)) <—– 12 roots, 6 errors

This is the picture of the above RS gen poly lists in graphical form:



- Coming up: Reed-Solomon functions explained

Elaborate REPL augmentation is needed to demystify the Reed-Solomon functions in this suite. The upcoming finished versions of RS output would be much, much better than these from GIMP:





- ‘Racket’ came from ‘Bracket’, allegedly

If you look at the total code size of this suite vis-a-vis the features, it is clear that Racket does a colossal job of compressing code size. Much of this gain comes from brackets – both the square kind and normal kind.

For instance, here is a ubiquitous recursive code snippet:

```
(let* ([d (degree g)][n (expt 2 d)]
[g-d (degree (cdr g))][p-l (take (cyclic-shift-mat (cdr g)) (- d g-d))]
[srt (reverse (append (reverse p-l) (reverse (cons (zeros d) (reverse (idn-mat d)))))])
(let rec ([i (length srt)][acc srt])
(if (>= i n)
acc
(if (positive? (car (last acc)))
(let ([ele (car (make-rect-k (list (binary-add-poly
(binary-mult-poly (cdr (last acc)) '(1 0))
(cdr g))) d))])
(if (member ele acc)
(rec (add1 i) acc)
(rec (add1 i) (append acc (list ele)))))
(rec (add1 i) (append acc (list (cyclic-left-shift (last acc) 1)))))))))
```

Just look at those brackets. The equivalent C code or Java code will have a separate line for each of the variables bound in each of the 'let*' clauses. It would look something like this:

```
int d = degree(g);
int n = exp(2, d);
...
...
another 5 lines.
```

Now for the main tail recursion, each 'if-else' clause has to be couched within brackets, so thats another expansion.

Then there's the range checks for each argument, checks for return values etc. In the end the same routine in C would be 60-70 lines. At least 5 times as long as the equivalent racket fragment.

For all the performance of imperative languages, the code size bloat is what introduces bugs – there is some recent study that pointed to smaller KLOC as being easier to maintain.

What's the solution then? Build the prototype in a functional programming language, test it thoroughly and then port it to C or Java.

PART TWO: How to view full-res versions of images posted here

This took a while to figure out but here it is:

1. Click 'View Image' on any image on a post.
2. In the URL of the displayed image, delete the portion after the file name extension. For instance in the 'More Progress <blah blah>' post below, remove the trailing ?w=450&h=254 part after the .png extension.
3. Hit enter.
4. The original, full resolution image will load now.

PART THREE: Colin McRae Rally 2005 on PS2 is totally irrelevant to this blog

But I’m going to write about it anyway. Because it’s the best game ever.

And this is why:

1. The ultra-realistic weather settings: sunny, cloudy and rainy weather settings. The Germany tracks are the most difficult in rain. The career mode gradually eases you into rainy weather by starting with sunny weather, then cloudy and so on.

2. The physics engine and the camera are beautifully integrated. While cornering, the camera has the same lag as a driver’s and navigator’s views would have. Hence the realism. While braking, the camera again follows an actual driver’s head movement. Some more realism injected.

3. All the advised driving tips work. You can sense your driving skill improve with application. The scandinavian flick was difficult to learn but the reward is watching your Citroen Xsara corner beautifully in a narrow stretch at 150 kph.

4. The rumble effect on the joystick is unbelievable. Greece circuits were a standout. You can hear the hard terrain hitting the axle.

- All further development on the suite will be in Clojure

Racket was a great win in terms of development, but Clojure’s widespread acceptance is hard to beat. The one minus about Clojure is the absence of a canonical IDE like DrRacket. But that will be a minor inconvenience since the command line interpreter seems to work fine.

I will also port to Clojure some specific examples from each of the prominent code categories such as Hamming, Reed-Muller, Binary BCH and Reed-Solomon. But it won't be generic, so that's something a Clojure expert can help me with and gain notoriety by getting their name mentioned here.

←Previous PageNext Page→

Indice ––––––––––
Blog at WordPress.com.

Channel Coding with Racket

- About
- Contact
- Miscellaneous test code
- source: Binary BCH, Reed-Solomon, Golay, Product codes
- Source: Hamming, Reed-Muller, Cyclic, and general linear block code properties
- Things to do
- Update
- Usage reference

Channel coding suite
Tumblr ↗️

- Reposting: changing defaults for polynomials, gen mats

If you want to use different generator polynomials, you'll need to shuffle the prim-polys-list variable. Then replace the original prim-polys-list variable in the source with the new one. This is how:

```
> (define poly-list (map (lambda(x) (shuffle x)) prim-polys-list))
> poly-list
'(((1 0 1 1))
((1 0 0 1 1))
((1 1 1 1 0 1) (1 0 0 1 0 1) (1 1 0 1 1 1))
((1 1 0 0 1 1 1) (1 1 0 1 1 0 1) (1 0 0 0 0 1 1))
((1 1 1 0 0 1 0 1)
(1 1 0 0 1 0 1 1)
(1 0 1 1 1 1 1 1)
(1 0 0 0 0 0 1 1)
(1 0 0 0 1 0 0 1)
(1 0 0 1 1 1 0 1)
(1 1 1 1 0 1 1 1)
(1 0 0 0 1 1 1 1)
(1 1 0 1 0 1 0 1))
((1 0 0 1 0 1 0 1 1)
(1 1 1 1 0 0 1 1 1)
(1 0 1 0 1 1 1 1 1)
(1 0 1 1 0 0 1 0 1)
(1 0 0 0 1 1 1 0 1)
(1 0 1 1 0 1 0 0 1)
(1 0 1 1 0 0 0 1 1)
(1 1 1 0 0 0 0 1 1))
((1 1 0 1 1 0 0 0 0 1)
(1 0 0 1 1 1 0 1 1 1)
(1 1 0 0 1 1 0 0 0 1)
(1 0 0 0 1 0 1 1 0 1)
(1 0 0 0 0 1 0 0 0 1)
(1 0 1 0 1 1 0 1 1 1)
(1 1 0 0 0 1 0 1 0 1)
(1 0 0 1 0 1 1 0 0 1)
(1 1 0 0 0 1 0 0 1 1))
((1 0 0 0 0 0 0 1 0 0 1)
(1 0 1 1 1 1 1 1 0 1 1)
```

(1 1 0 1 0 0 0 1 0 0 1)
(1 1 0 1 1 0 1 1 1 1 1)
(1 1 0 0 0 0 1 0 1 0 1)
(1 0 1 0 0 1 1 0 0 0 1)
(1 0 1 0 0 0 0 1 1 0 1)
(1 0 0 0 0 0 1 1 0 1 1)
(1 0 1 0 0 1 0 0 0 1 1)
(1 1 1 0 1 0 0 1 1 0 1)
(1 1 1 1 1 1 1 1 0 0 1)
(1 0 0 0 1 1 0 1 1 1 1))
((1 1 1 1 0 0 0 0 1 0 1 1)
(1 1 1 1 0 0 1 1 0 0 0 1)
(1 1 0 0 0 0 0 0 1 1 0 1)
(1 0 1 0 1 0 1 0 1 0 1 1)
(1 0 0 0 0 0 0 0 0 1 0 1)
(1 0 0 0 1 0 0 0 1 1 0 1)
(1 0 0 1 0 1 1 1 0 0 1 1)
(1 0 0 0 0 0 1 0 1 1 0 1)
(1 0 0 0 0 1 1 0 0 0 1 1)
(1 0 0 1 0 0 1 0 0 1 0 1))
((1 0 1 1 1 0 1 1 1 0 1 0 1)
(1 1 0 1 1 1 1 0 0 0 0 0 1)
(1 1 1 1 1 1 0 1 1 1 0 1 1)
(1 0 0 0 1 1 1 0 1 1 1 1 1)
(1 0 0 1 0 0 0 0 0 1 1 0 1)
(1 1 0 1 1 1 0 1 0 0 1 1 1)
(1 0 0 0 0 0 1 0 1 0 0 1 1)
(1 1 1 0 0 0 0 1 0 0 1 1 1)
(1 1 1 0 1 0 1 0 1 1 0 1 1)
(1 1 0 0 0 0 1 0 1 0 1 1 1))
((1 0 1 0 0 1 1 0 0 1 0 0 1 1)
(1 0 0 0 1 0 1 0 1 1 1 1 1 1)
(1 0 1 1 1 1 1 1 1 1 1 1 1 1)
(1 1 0 0 0 1 1 1 1 0 0 0 0 1)

(1 0 1 1 0 0 1 0 0 0 1 0 1 1)
(1 1 0 1 0 1 1 0 0 1 1 1 0 1)
(1 0 1 1 0 1 1 1 1 1 1 1 0 1)
(1 0 0 1 1 1 0 1 0 0 1 1 1 1)
(1 0 0 0 1 1 1 0 1 0 0 0 1 1)
(1 0 0 1 1 0 1 0 1 1 0 0 0 1)
(1 1 0 1 1 1 1 0 0 1 1 0 0 1)
(1 1 0 1 0 1 1 1 1 0 0 0 1 1)
(1 0 0 0 0 0 0 0 0 1 1 0 1 1)))

You need to copy this variable and replace the original with it.

Alternatively you can define your own functions:

> (your-BCH-gp 5 4)
‘(1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1)
> (BCH-gp 5 4)
‘(1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1)
> (BCH-gp 4 3)
‘(1 0 1 0 0 1 1 0 1 1 1)
> (your-BCH-gp 4 3)
‘(1 0 1 0 0 1 1 0 1 1 1)

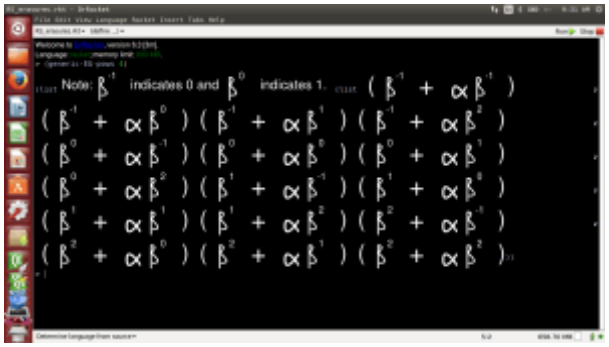
> (your-BCH-gm 5 4)
‘((1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0)
(0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0)
(0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0 0)
(0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0 0)
(0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0 0)
(0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0 0)
(0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0 0)
(0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0 0)
(0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 0 1 1 0 0 0 1 0 1))
> (BCH-gm 5 4)
‘((1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0)
(0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0)
(0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0)
(0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0)
(0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0)
(0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0)
(0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0)
(0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0)
(0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1))

- A programming blog is the best. You can keep dumping code output instead of actually writing something

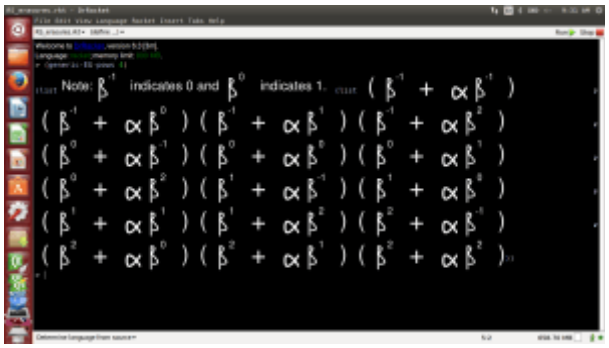
There might even be a way to design GAMES based on channel coding. Just. Think. Almost exciting isn’t it.
Worse, it might be made a role-playing game. You can choose your role as the encoder, channel or the decoder. The channel obviously keeps shooting errors, the encoder and decoder correct it etc etc.

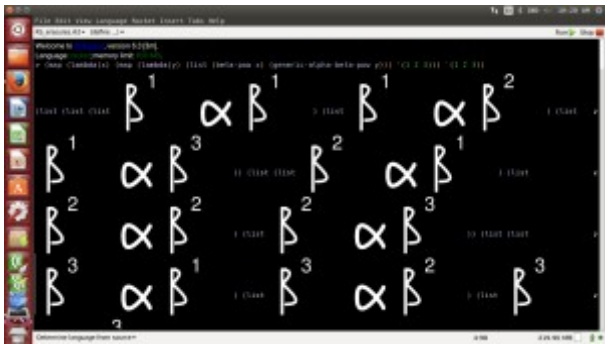
- More progress displaying Euclidean Geometry symbols

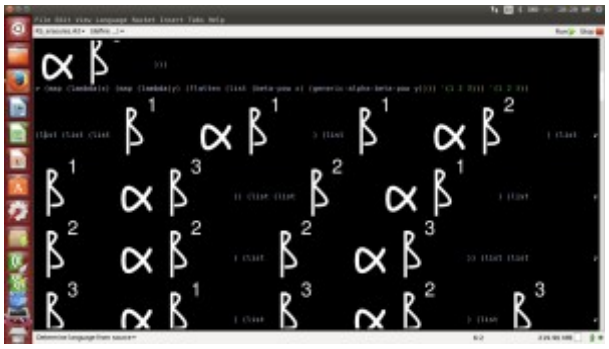
This is turning out to be more complicated than anticipated but I'm glad to report progress:

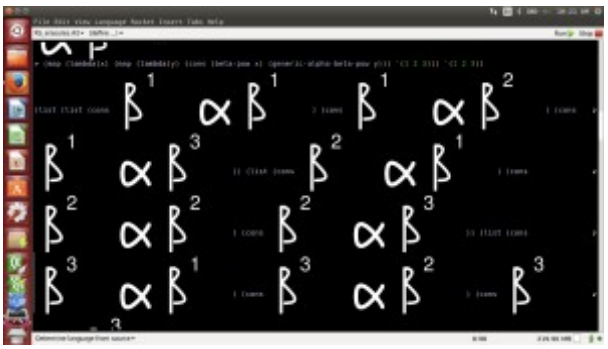


If want to see all the intermediate steps, here they are:

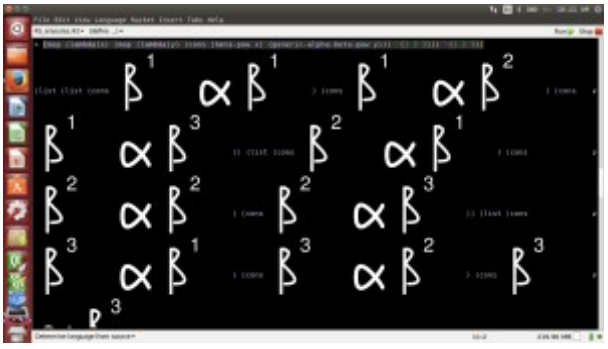




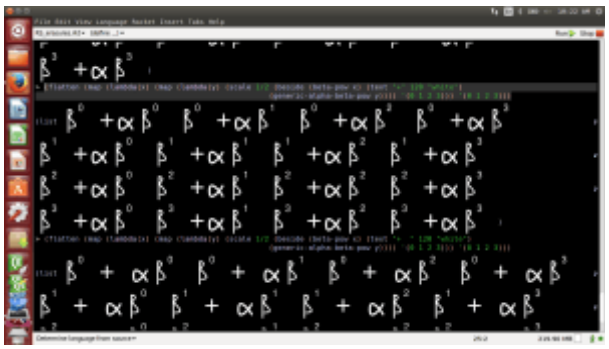














- To be clear, only block coded modulation is supported

Trellis coded modulation may never be supported in this suite because I haven't figured out how to fit convolutional codes into this architecture (or the glaring lack of it).
Also, there were some problems with the function contracts in the coded modulation functions. If you experienced it, you need to change the return type to (listof any/c) from the current any/c.

Sample output after fixing the function contracts:

```
> (sampled-unit-sin 16 16)
'(0
0.3826834323650898
0.7071067811865475
0.9238795325112867
1.0
0.9238795325112867
0.7071067811865476
0.3826834323650899
1.2246467991473532e-16
-0.38268343236508967
-0.7071067811865475
-0.9238795325112865
-1.0
-0.9238795325112866
-0.7071067811865477
-0.3826834323650904)
```

```
> (sampled-unit-cos 16 16)
'(1
0.9238795325112867
0.7071067811865476
0.38268343236508984
6.123233995736766e-17
-0.3826834323650897
-0.7071067811865475
-0.9238795325112867
-1.0
-0.9238795325112868
-0.7071067811865477
-0.38268343236509034
-1.8369701987210297e-16
0.38268343236509
0.7071067811865474
0.9238795325112865)

> (sampled-unit-cos 14 31)
'(1
0.9795299412524945
0.9189578116202306
0.8207634412072763
0.6889669190756866
0.5289640103269624
0.3473052528448203
0.1514277775045767
-0.0506491688387126 4
-0.2506525322587206 4
-0.4403941515576344
-0.6121059825476629
-0.7587581226927909
-0.8743466161445821)
```

> (sampled-unit-sin 14 31)
‘(0
0.20129852008866006
0.39435585511331855
0.5712682150947923
0.7247927872291199
0.8486442574947509
0.9377521321470804
0.9884683243281114
0.9987165071710528
0.9680771188662043
0.8978045395707416
0.7907757369376985
0.6513724827222223
0.48530196253108104)

This output and usage for other functions is partly (?) self-explanatory. In other words, please look at the source and figure it out yourself.

- Support available for implementing coded modulation

Here’s a list of functions you can use for coded modulation:
[sampled-fixpt-sin (positive? positive? . -> . any/c)]

[sampled-fixpt-cos (positive? positive? . -> . any/c)]

[sampled-fixpt-sin-fq (positive? positive? positive? . -> . any/c)]

[sampled-fixpt-cos-fq (positive? positive? positive? . -> . any/c)]

[fixpt-cos-freqs (positive? . -> . any/c)]

[fixpt-sin-freqs (positive? . -> . any/c)]

[plot-sinc (positive? . -> . any/c)]

[plot-sinc2 (positive? . -> . any/c)]

- Soft-decision decoding is a programmer's paradise

Reliability-based soft decision decoding algorithms especially are almost a pure programmer's domain if the underlying algebraic structure is elegant and not too complicated. For instance, this is a snippet from an under-development file:

```
(define RS_63_57_all_1_err_syndromes
; (map (lambda(x) (flatten x))
'(((1 0 0 0 0 1) (1 1 0 0 0 1) (1 1 1 0 0 1) (1 1 1 1 0 1) (1 1 1 1 1 1) (1 1 1 1 1 0))
((0 0 0 0 0 1) (0 0 0 0 0 1) (0 0 0 0 0 1) (0 0 0 0 0 1) (0 0 0 0 0 1) (0 0 0 0 0 1))
...
...

((0 1 0 0 0 0) (0 0 1 1 0 0) (0 0 0 1 0 1) (0 1 0 0 1 1) (1 1 1 1 0 0) (0 1 0 0 0 1))
((1 0 0 0 0 0) (1 1 0 0 0 0) (1 0 1 0 0 0) (1 1 1 1 0 0) (1 0 0 0 1 0) (1 1 0 0 1 1))
```

Basically it lists all the 6 RS syndromes powers corresponding to all the 1-symbol errors in the RS(63, 57) code. A soft decision decoder can use this as a lookup table to check if there is a 1-symbol error. Even if no match is found, one can use to this is check for 2 symbol errors by iteratively adding the syndromes and comparing it with the syndromes of the received vector.

This is how: PSEUDOCODE

(if (syndrome S1 of received vector == (syndrome S1 of element n11 of RS_63_57_all_1_err_syndromes) + (syndrome S1 of element n12 of RS_63_57_all_1_err_syndromes)))

—> proceed to repeat above step for matching S2 of received vector's syndrome with that of n21 and n22 of the lookup table, if match found

———> proceed till S1 through S6 matches are found.

More on this later.

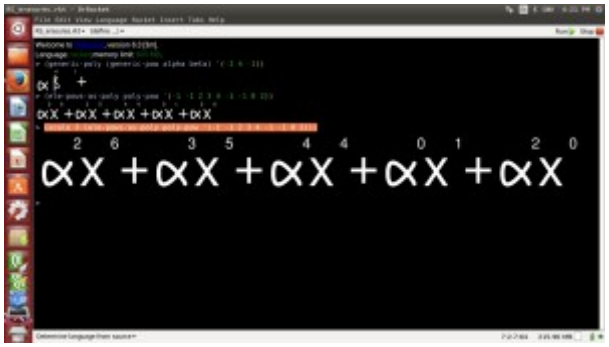
- Converting RS poly representation in the code to displayable images (for teaching)

Here's some screenshots from my ongoing work to convert the representation of RS polynomials and EG symbols in the source code to displayable images. Uses: elucidation and education.
1. This function displays RS polynomials as images:

> (ele-pows-as-poly poly-pow '(-1 -1 2 3 4 -1 -1 0 2))

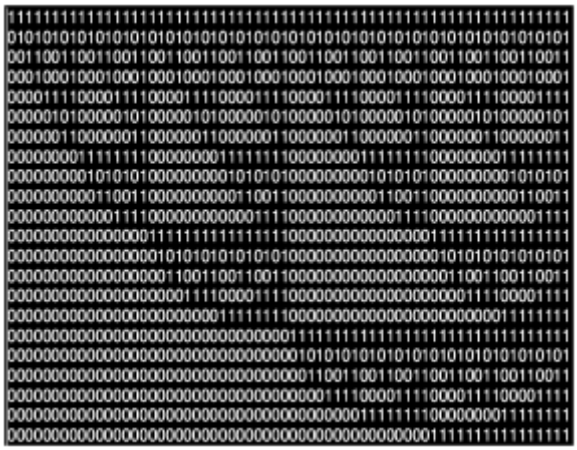
You can scale it to display this image:

> (scale 3 (ele-pows-as-poly poly-pow '(-1 -1 2 3 4 -1 -1 0 2)))



2. This function takes in a binary matrix and displays it as a scaled image:

> (scale 3/4 (matrix-2-image (RM-genmat 3 6)))



- Time performance of RM MLD decoder for real world vectors

Here noise vectors are added to all rows of RM gen mat.

> (define gm (RM-genmat 3 7))
> (define noi-v (car (map (lambda(x) (shuffle (append (zeros 114) x)))
(gen-nbit-patterns 14 8))))
> (define gm-nv (add-noise-vector gm noi-v))
> (define dv (time (decode-RM-vectors gm-nv (RM-genmat 3 7))))
cpu time: 51113 real time: 51476 gc time: 16716
> (sub-mat dv 0 40 0 55)
‘((1 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0))
> (sub-mat dv 40 64 10 64)
‘((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)

(0 0 0

> (equal? dv (idn-mat 128))
#f
>

- Time performance of RM MLD decoder for simple vectors: all-zeros and gen mat rows

> (define dv (time (decode-RM-vectors (RM-genmat 3 7) (RM-genmat 3 7))))
cpu time: 48589 real time: 48599 gc time: 15403
It took nearly 50 seconds to decode all the vectors that make up th generator matrix.

> (length dv)
64
> (width dv)
64
> (equal? dv (idn-mat (length dv)))
#t
> (define dv2 (time (decode-RM-vectors (RM-genmat 3 6) (RM-genmat 3 6))))
cpu time: 11095 real time: 11107 gc time: 6930
> dv2
‘((1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1)

(0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0

(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1))
> (equal? dv2 (idn-mat (length dv2)))
#t
>

> (define dv0 (map (lambda(x) (shuffle (append (zeros 52) x))) (gen-nbit-patterns 12 6)))
> (sub-mat dv0 0 15 0 25)
‘((0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0)
(0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0)
(0 1 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0)
(0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1)
(0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0))
> (sub-mat dv0 0 15 0 55)
‘((0 0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0)
(0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1)
(0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1)
(0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0)
(0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
> (define dv (time (decode-RM-vectors (take dv0 10) (RM-genmat 3 6))))
cpu time: 1081 real time: 1080 gc time: 69
> dv
‘((0 0 0 0 0 1 0 0 1 1 0 0 1 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0)
(0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)

(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1))
> (define dv0 (map (lambda(x) (shuffle (append (zeros 52) x))) (gen-nbit-patterns 12 5)))
> (sub-mat dv0 0 15 0 55)
‘((0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0)
(0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0)
(0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0)
(0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0))
> (define dv (time (decode-RM-vectors (take dv0 10) (RM-genmat 3 6))))
cpu time: 7147 real time: 7162 gc time: 6016
> dv
‘((0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
1 0 0)
(0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0
0 1 1)
(0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0
0 0 1)
(0 0 0 0 1 1 1 0 0 0 0 0 1 1 1 0 0 0 0 1 1 1 0 1 1 1 0 0 0 0 1 1 1 0 1 1 1 0 0
0 0 0)
(1 1 1 0 0 0 0 1 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1
1 0 0)
(1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 1 1 0 0 1 1 0 1 1 1 1 0 0 1 1 0 0
1 0 1)
(0 1 0 1 0 1 1 0 0 1 1 1 0 1 1 0 1 1 0 0 0 0 0 1 0 0 0 1 0 1 1 0 1 1 1 1 0 1 0
1 0 1)
(0 1 0 1 0 1 0 1 1 0 1 0 1 0 1 0 1 0 1 1 0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0)
(1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 0 1 0 0 0
0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
0 0 0))
> (define dv0 (map (lambda(x) (shuffle (append (zeros 52) x))) (gen-
nbit-patterns 12 4)))
> (sub-mat dv0 0 15 0 55)
‘((0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1
0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0)
(1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0)
(0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0)
(1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0))
> dv
‘((0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
1 0 0)
(0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0
0 1 1)
(0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0
0 0 1)
(0 0 0 0 1 1 1 0 0 0 0 0 1 1 1 0 0 0 0 1 1 1 0 1 1 1 0 0 0 0 1 1 1 0 1 1 1 0 0
0 0 0)

(1 1 1 0 0 0 0 1 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 0 0)
(1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 1 1 0 0 1 1 0 1 1 1 1 0 0 1 1 0 0 1 0 1)
(0 1 0 1 0 1 1 0 0 1 1 1 0 1 1 0 1 1 0 0 0 0 0 1 0 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 1)
(0 1 0 1 0 1 0 1 1 0 1 0 1 0 1 0 1 0 1 1 0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 0 1 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0))
> (define dv (time (decode-RM-vectors (take dv0 10) (RM-genmat 3 6))))
cpu time: 1074 real time: 1072 gc time: 58
> dv
‘((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)

(0 0 0

> (define dv0 (map (lambda(x) (shuffle (append (zeros 110) x))) (gen-nbit-patterns 18 6)))
> (define dv (time (decode-RM-vectors (take dv0 10) (RM-genmat 3 7))))
cpu time: 12277 real time: 12285 gc time: 7015
> (all-zeros? (flatten dv))
#t
> (sub-mat dv 0 15 0 55)
. . take: index is too large for list
index: 15
list: '((0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0)
(0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 1 1 0 0 0 0 0)
(1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 1 0 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 1
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0))
>

> (define dv0 (map (lambda(x) (shuffle (append (zeros 110) x))) (gen-nbit-patterns 18 8)))
> (sub-mat dv0 0 10 0 55)
‘((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0
0 0 0 1 0 0 0 0 0 0 1 0 0 0 1 0)
(0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0))
> (define dv (time (decode-RM-vectors (take dv0 10) (RM-genmat 3 7))))
cpu time: 5931 real time: 5930 gc time: 446
> (all-zeros? (flatten dv))
#t
> (sub-mat dv 0 10 0 55)
‘((0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)

(0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```
Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.
> (define i (info-bits 18))
> (size i)
9437184
> (length i)
262144
> (define t (time (transpose i)))
cpu time: 5468 real time: 5467 gc time: 2308
```

So it took about 5.5 seconds for a 10 million element list.

2. Encoding RM codes:

```
> (define t1 (time (encode-RM-all-cw 2 4)))
cpu time: 127 real time: 126 gc time: 37
> (size t1)
65536
> (take (shuffle t1) 25)
'((1 0 0 0 1 0 1 1 0 0 1 0 0 0 0 1)
(1 0 1 0 1 1 1 1 1 0 1 0 0 0 0 0)
(1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 1)
(0 1 0 1 1 1 0 0 1 0 1 0 1 1 0 0)
(0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0)
(1 0 1 0 1 1 0 0 0 1 1 0 1 1 1 1)
(0 1 0 0 1 1 1 0 1 0 1 1 0 0 0 1)
(1 0 0 0 1 0 1 1 1 0 1 1 0 1 1 1)
(0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 0)
(1 1 1 0 1 0 0 0 1 0 0 0 1 1 1 0)
(1 1 1 0 0 1 0 0 1 0 1 1 1 1 1 0)
(1 0 1 0 0 1 1 0 1 1 1 1 0 0 1 1)
(1 1 0 0 0 1 0 1 0 0 1 1 1 0 1 0)
(1 1 0 0 1 0 0 1 0 1 0 1 1 1 1 1)
(1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1)
(1 0 1 1 0 1 0 0 1 1 0 1 0 0 1 0)
(1 1 0 0 1 1 0 0 1 1 0 0 0 0 1 1)
(1 1 0 1 1 0 0 0 0 1 0 0 0 0 0 1)
(0 1 1 1 1 0 0 0 1 1 1 0 1 1 1 0)
(0 0 1 0 0 0 1 0 1 1 0 1 0 0 1 0)
(1 0 0 1 1 0 0 1 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1)
(0 0 1 0 0 1 0 0 1 1 0 1 1 0 1 1)
(0 1 1 0 1 0 1 0 0 1 0 1 1 0 0 1)
(1 0 1 1 1 0 1 1 0 0 1 0 1 1 0 1))
> (length (RM-genmat 2 5))
16
> (length (RM-genmat 3 5))
26
```

```
> (length (RM-genmat 3 4))
15
```

```
> (RM-genmat 3 5)
'((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 1)
(0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 1)
(0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1)
(0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1)
(0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 1)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 1 0 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1))
> (RM-genmat 2 5)
'((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 1)
(0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 1)
(0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1)
(0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1)
```

(0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1))
> (RM-genmat 3 5)
‘((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1)
(0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1)
(0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1)
(0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1)
(0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1)

(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1))

> (define t2 (time (encode-RM-all-cw 2 5)))
cpu time: 4883 real time: 4884 gc time: 1418
> (define t1 (time (encode-RM-all-cw 2 4)))
cpu time: 91 real time: 90 gc time: 40
> (define t2 (time (encode-RM-all-cw 2 5)))
cpu time: 4920 real time: 4915 gc time: 1501

It takes about 5 seconds for a larger block RM.

> (size t2)
4194304
> (length t2)
65536

NOTE: RM(2,5) above has the same number of elements as RM(2,4) but codewords twice as long, plus this time includes generating the RM(2,5) generator matrix as well, but it’s still disproportionate obviously.

```
> (take (shuffle t2) 15)
'((0 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 1 1 1 0 0 1 0 0 0 1 1 1 1 1 0 1)
(1 1 0 0 0 1 1 0 0 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 0 1 1 0 0 0 1 1)
(0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 0 1 1 1 0 0 1 0)
(0 0 1 1 1 1 1 1 0 1 0 1 1 0 0 1 0 1 0 1 1 0 0 1 1 1 0 0 0 0 0 0)
(0 1 0 1 1 1 1 1 0 1 1 0 1 1 0 0 0 1 0 1 1 1 1 1 0 1 1 0 1 1 0 0)
(0 0 1 0 1 0 1 1 0 0 1 0 0 1 0 0 0 0 0 1 1 0 0 0 1 1 1 0 1 0 0 0)
(1 1 1 0 1 1 0 1 0 1 1 1 1 0 1 1 1 0 0 0 1 0 1 1 0 0 0 1 1 1 0 1)
(0 1 0 0 0 0 1 0 1 1 1 0 1 0 0 0 1 1 0 1 0 1 0 0 0 1 1 1 1 1 1 0)
(0 1 1 0 1 1 1 1 1 1 1 1 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 0 0 0 1 1)
(1 1 0 1 0 1 1 1 1 1 0 1 1 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 1 1 1 0)
(1 1 0 0 0 0 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0)
(1 1 0 1 0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 1 1 0 1 1 1 0 0 0 0 0 1 0)
(1 0 0 0 1 0 0 0 1 0 1 1 1 0 1 1 0 1 1 1 1 0 0 0 0 1 0 0 1 0 1 1)
(0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0 1 0 1 1 1 0 0 1 0 1 0)
(0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0 1 1 1 0 1 1))
```

```
> (define t3 (time (transpose t2)))
cpu time: 2219 real time: 2223 gc time: 872
```

As seen above, transpose scales down linearly from the earlier example which took 5.5 seconds for 10 million elements, and the one above took half the time for roughly 4.2 million elements.

←Previous PageNext Page→

Indice ––––––––––

Blog at WordPress.com.

- source: Binary BCH, Reed-Solomon, Golay, Product codes
- Source: Hamming, Reed-Muller, Cyclic, and general linear block code properties
- Things to do
- Update
- Usage reference

Channel coding suite
Tumblr ↗️

- Time performance for BCH generator matrix generation

  All cpu time figures are in milliseconds. For larger blocks it’s considerably slower because it generates, factorizes and solves all the conjugate polynomials EVERY SINGLE TIME. This can be avoided if you store the generator matrices.

```
> (define t (time (BCH-gm 5 5)))
cpu time: 12 real time: 12 gc time: 0
> t
'((1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 0 0 0)
(0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 0 0)
(0 0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0 0)
(0 0 0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0 0)
(0 0 0 0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0)
(0 0 0 0 0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 0 0 0 1 0 0 1 1 1))
> (define t (time (BCH-gm 5 3)))
cpu time: 10 real time: 10 gc time: 0
> t
'((1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1))
> (define t (time (BCH-gm 6 4)))
cpu time: 37 real time: 36 gc time: 0
> (define t (time (BCH-gm 8 7)))
cpu time: 666 real time: 666 gc time: 28
```

> (define t (time (eucl-dec-end-2-end-test g3 6)))
cpu time: 10061 real time: 10053 gc time: 300

- Time performance for nCr generators

Here's an idea of the size of the vectors:

> (num-error-patterns 15 9)
5005
> (num-error-patterns 31 9)
20160075

> (num-error-patterns 15 11)
1365
> (num-error-patterns 31 10)
44352165
> (num-error-patterns 31 6)
736281
> (num-error-patterns 18 11)
31824
> (num-error-patterns 18 9)
48620
> (num-error-patterns 21 9)
293930

> (num-error-patterns 20 9)
167960
> (define t (time (gen-nbit-patterns 18 9)))
cpu time: 4899 real time: 4898 gc time: 2859

> (length t)
48620

So it takes about 5 seconds to generate all the 18C9 patterns.

Here's something larger:

```
> (num-error-patterns 20 9)
167960
> (define t (time (gen-nbit-patterns 20 9)))
cpu time: 54919 real time: 55026 gc time: 44693
```

It took FIFTY FIVE SECONDS to generate all the 167960 20C9 combinations.

> (size t) (length t)
167960
> (flat-length t) (take (reverse t) 25)
‘((1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 1 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 1 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 1 0 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 1 0 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 0 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 1 0)
(1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1)
(1 1 1 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 1 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 1 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 1 0 0 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 1 0 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 0 1 0 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 1 0 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 1 0)
(1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 1)
(1 1 1 1 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0)
(1 1 1 1 1 1 1 0 0 1 0 1 0 0 0 0 0 0 0))

- Time performance for all n-tuples generator

Generating n-tuples is hard on CPU and on memory, and the latter can run out if the word lengths are large enough. Here’s a few I recorded, all figures are in milliseconds so any 5-digit figures equates to an interminable wait:

```
Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.
> (define t (time (info-bits 13)))
cpu time: 47 real time: 47 gc time: 0
> (define t (time (info-bits 16)))
cpu time: 1064 real time: 1067 gc time: 541

> (take (reverse t) 25)
'((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1))
```

Here’s something even bigger:

```
> (define t (time (info-bits 19)))
cpu time: 24995 real time: 25038 gc time: 19487
```

It takes 25 seconds to create all the 2^19 patterns. A part of the output:

```
> (take (reverse t) 25)
‘((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 1)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 0)
(1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1))
```

(info-bits 21) and above will overrun 800MB of memory.

- Euclidean decoding algorithm for RS codes can't correct more than 2 errors?

This is a problem I faced with Euclidean decoding. First, some decoding of random 2-error RS vectors:
> (eucl-dec-end-2-end-test g2 2)
'((((26 15 5)
((..snip)
(1 9)
(30 22) <– this is error locator polynomial))
(...snip)
(12 1 -1 16 13 26 25 30 18 22 22 23 7 6 29 12 3 22 9 6 7 1 11 -1 -1 -1 -1 -1 -1 -1 -1)
(30 29 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8)
(1 -1 -1 -1 -1 -1 -1 -1 29 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1)
(24 1 -1 16 13 26 25 30 10 22 22 23 7 6 29 12 3 22 9 6 7 1 11 -1 -1 -1 -1 -1 -1 -1 -1)
(...snip) (1 29) (snip)

> (eucl-dec-end-2-end-test g2 2)
‘((((0 1 5)
((...snip)
(19 7)))
(...snip)
(16 13 21 13 28 27 6 -1 -1 11 0 -1 15 19 16 7 16 0 11 16 24 20 28 20 4 3 13 -1 -1 -1 -1)
(30 29 28 27 26 25 24 21 20 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4)
(19 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 8 -1 -1 -1 -1 -1 -1 -1)
(16 13 21 13 28 27 6 -1 -1 11 0 19 15 19 16 7 16 0 11 16 24 20 28 28 4 3 13 -1 -1 -1 -1)(...snip)
(19 8) (20 10))
...snip)

As you can see the patterns in bold, it’s able to resolve 2 error locations AND correct error values.

Now I check for 3 or more errors with the same g2:

> (eucl-dec-end-2-end-test g2 3)
‘((((15 25 13 8)
(...snip)
(18 17 3) <– correct error locator for this pattern: (2 2 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 4 -1 -1 -1)

But error values: '(13 8 12). This is WRONG.

I'll ponder on the theoretical reasons for this on a subsequent post.

- The ‘linear’ in linear block codes

The first time I read the theorem that ‘the minimum distance of a code is the weight of the minimum weight codeword’, I hadn’t fully understood that the linearity property led directly to the theorem. But I was using the above result with the following minimum distance functions:

```
Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.

; — encodes all codewords and finds least weight codeword
> (min-dist-gm (RM-genmat 2 5))
8

(RM-genmat 2 5)
‘((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1)
(0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1)
(0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1)
(0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1)
(0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1)
(0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1))
```

And then there is a min-dist function for arbitrary sets of codewords which may or may not have been generated from a matrix

> (min-dist (info-bits 4))
1
> (info-bits 4)
‘((0 0 0 0)
(0 0 0 1)
(0 0 1 0)
(0 0 1 1)
(0 1 0 0)
(0 1 0 1)
(0 1 1 0)
(0 1 1 1)
(1 0 0 0)
(1 0 0 1)
(1 0 1 0)
(1 0 1 1)
(1 1 0 0)
(1 1 0 1)
(1 1 1 0)
(1 1 1 1))

Here it’s all possible 4-tuples which is considered as the ‘code’.

- Very limited success finding cyclic RM generator matrix format

The simplest cyclic RM genmat with the last column removed from (RM-genmat 2 3):
> (cyclic-RM-genmat 2 3)
‘((1 1 1 1 1 1 1) (0 1 0 1 0 1 0) (0 0 1 1 0 0 1) (0 0 0 1 0 0 0) (0 0 0 0 1 1 1) (0 0 0 0 0 1 0) (0 0 0 0 0 0 1))

This is the only special case that is cyclic without row or column operations:

> (check-cyclicity-gm (cyclic-RM-genmat 2 3))
#t

Now these are all the other RM gen mats that didn't work for cyclicity:

> (cyclic-RM-genmat 2 4)
'((1 1 1 1 1 1 1 1 1 1 1 1 1 1 1)
(0 1 0 1 0 1 0 1 0 1 0 1 0 1 0)
(0 0 1 1 0 0 1 1 0 0 1 1 0 0 1)
(0 0 0 1 0 0 0 1 0 0 0 1 0 0 0)
(0 0 0 0 1 1 1 1 0 0 0 0 1 1 1)
(0 0 0 0 0 1 0 1 0 0 0 0 0 1 0)
(0 0 0 0 0 0 1 1 0 0 0 0 0 0 1)
(0 0 0 0 0 0 0 0 1 1 1 1 1 1 1)
(0 0 0 0 0 0 0 0 0 1 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 0 1 1 0 0 1)
(0 0 0 0 0 0 0 0 0 0 0 0 1 1 1))

> (check-cyclicity-gm (cyclic-RM-genmat 2 4))
#f
> (check-cyclicity-gm (cyclic-RM-genmat 1 4))
#f

I have done several column operations like random swapping (check the first PDF for corresponding functions) but I have not tried adding an odd number of columns. Maybe that'll work. I'll post if it does.

- Continued from previous post: attempts with GIMP make-do

So what we need is a proper library that draws at least the few Greek symbols that GF arithmetic uses. Else we'll have create endless images like this terrible one:



The library needs to draw the powers of alpha and X as text, plus the + symbol. If you want to see you name mentioned here but you're not ready use these suite's functions yet, this is your chance to write a standalone library.

- Need for a Racket library to display BCH polynomials with Galois Field coefficients

First, this how this suite represents RS polynomials at present:
Example 1: '(3 -1 4 -1 5) stands for (alpha^3 * X^4) + (alpha^4 * X^2) + alpha^5. Missing coefficient powers of X are represented as -1's. The power of X is the position of the element in the list , with the most significant power first.

Example 2: '(0 0 2 5 -1) stands for X^4 + X^3 + (alpha^2 * X^2) + (alpha^5 * X). Any 0 value in the list indicates alpha^0 (= 1) at that position in the list.

- Extensions to the binary search tree in Racket Guide

This is more to demonstrate the power of Racket UI but it can be a useful template to build trees of different structures. Some images of code output: 
Write to me for the rkt source files. There are four files and I'd rather mail them in the native format than export them into a single PDF and make you do the reassembly at your end.

- Relax, hardly 610+ of the 635+ functions are undocumented

  At the rate of 10 functions or less a day, the whole documentation will be done in a short period of, well, several months.

←Previous PageNext Page→

Indice ––––––––––

Blog at WordPress.com.

Channel Coding with Racket

- About
- Contact
- Miscellaneous test code
- source: Binary BCH, Reed-Solomon, Golay, Product codes
- Source: Hamming, Reed-Muller, Cyclic, and general linear block code properties
- Things to do
- Update
- Usage reference

Channel coding suite

Tumblr ↗️

- The print poly function

Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.
> (printable-poly '(0 0 1 0 0 0 0 0 0 1 0 1 0 0 0))
'((0 * x ^ 14)
(0 * x ^ 13)
(1 * x ^ 12)
(0 * x ^ 11)
(0 * x ^ 10)
(0 * x ^ 9)
(0 * x ^ 8)
(0 * x ^ 7)
(0 * x ^ 6)
(1 * x ^ 5)
(0 * x ^ 4)
(1 * x ^ 3)
(0 * x ^ 2)
(0 * x ^ 1)
(0 * x ^ 0))
It's admittedly clumsy and maybe it should leave out zero coefficient powers, so why don't you take that as an exercise.

Usage for RS polynomials:

> p1
'(3 1 10 3 6 6 13 8 4 8 10 3 1 5 9)
> (printable-poly p1)
'((3 * x ^ 14)
(1 * x ^ 13)
(10 * x ^ 12)
(3 * x ^ 11)
(6 * x ^ 10)
(6 * x ^ 9)
(13 * x ^ 8)
(8 * x ^ 7)
(4 * x ^ 6)
(8 * x ^ 5)
(10 * x ^ 4)
(3 * x ^ 3)
(1 * x ^ 2)
(5 * x ^ 1)
(9 * x ^ 0))

- Binary BCH textbook example

For test vector '(0 0 1 0 0 0 0 0 0 1 0 1 0 0 0), these are the remainders. The last one seems wrong first, but it resolves to alpha^10 all the same.

Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.
> (BCH-cw-remainders '(0 0 1 0 0 0 0 0 0 1 0 1 0 0 0) 4 3)
'((1) (1 1 0 1) (1 1))
>

- Note: all binary polynomials are MSB first and RS polys, most significant symbol first

This is the only departure from the Lin-Costello nomenclature where it's LSB and least significant symbol first respectively.

- Binary BCH: gen poly and gen mat

Welcome to DrRacket, version 6.3 [3m].
Language: racket; memory limit: 800 MB.
first parameter: polynomial degree
second parameter: number of errors

> (BCH-gp 4 3)
'(1 0 1 0 0 1 1 0 1 1 1)

> (BCH-gm 5 4)
'((1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0)
(0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0)
(0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0)
(0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0)
(0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0)
(0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 0)
(0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0)
(0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0)
(0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0 0)
(0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1 0)
(0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 1 1 0 1 0 1 0 1))

- This suite follows the Shu Lin-Daniel Costello textbook almost page by page

The best way to confirm the accuracy of this software is to feed it the examples in the Lin-Costello textbook. Most of the functions have a default version which use the generator matrices, parity matrices and test vectors provided in the book.

I'll post shortly on how to choose different parameters (from the default ones) for these functions.

- Coming soon: Finite Geometry codes

Hereafter I'll only post function outlines and not readily usable code. This is to encourage users to create their own version of the published code (see source pages above).

←Previous Page

Indice ––––––––––

Blog at WordPress.com.